

Appendix A: App Notes

Attaching a 7 segment LED via GPIO

This example code shows how to interface the Raspberry Pi Pico to a generic 7 segment LED device. It uses the LED to count from 0 to 9 and then repeat. If the button is pressed, then the numbers will count down instead of up.

Wiring information

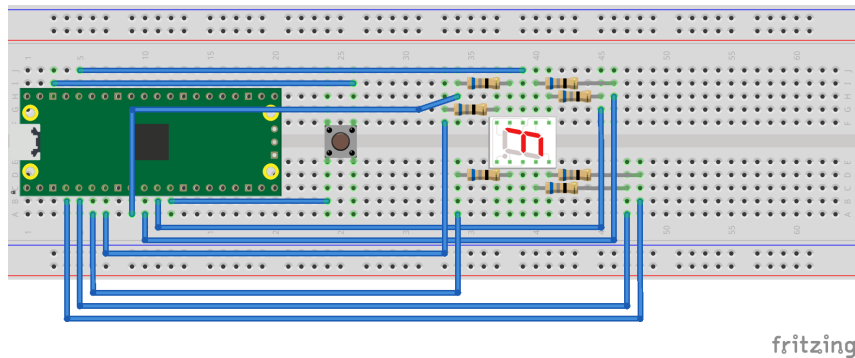
Our 7 Segment display has pins as follows.

```
--A--
F  B
--G--
E  C
--D--
```

By default we are allocating GPIO 2 to segment A, 3 to B etc. So, connect GPIO 2 to pin A on the 7 segment LED display and so on. You will need the appropriate resistors (68 ohm should be fine) for each segment. The LED device used here is common anode, so the anode pin is connected to the 3.3v supply, and the GPIOs need to pull low (to ground) to complete the circuit. The pull direction of the GPIOs is specified in the code itself.

Connect the switch to connect on pressing. One side should be connected to ground, the other to GPIO 9.

Figure 9. Wiring Diagram for 7 segment LED.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/gpio/hello_7segment/CMakeLists.txt

```
1 add_executable(hello_7segment
2     hello_7segment.c
3 )
4
5 # pull in common dependencies
6 target_link_libraries(hello_7segment pico_stdlib)
7
```

```

8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(hello_7segment)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(hello_7segment)

```

hello_7segment.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/gpio/hello_7segment/hello_7segment.c

```

1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "hardware/gpio.h"
10
11 /*
12  Our 7 Segment display has pins as follows:
13
14  --A--
15  F    B
16  --G--
17  E    C
18  --D--
19
20  By default we are allocating GPIO 2 to segment A, 3 to B etc.
21  So, connect GPIO 2 to pin A on the 7 segment LED display etc. Don't forget
22  the appropriate resistors, best to use one for each segment!
23
24  Connect button so that pressing the switch connects the GPIO 9 (default) to
25  ground (pull down)
26 */
27
28 #define FIRST_GPIO 2
29 #define BUTTON_GPIO (FIRST_GPIO+7)
30
31 // This array converts a number 0-9 to a bit pattern to send to the GPIOs
32 int bits[10] = {
33     0x3f, // 0
34     0x06, // 1
35     0x5b, // 2
36     0x4f, // 3
37     0x66, // 4
38     0x6d, // 5
39     0x7d, // 6
40     0x07, // 7
41     0x7f, // 8
42     0x67, // 9
43 };
44
45 /// \tag::hello_gpio[]
46 int main() {
47     stdio_init_all();
48     printf("Hello, 7segment - press button to count down!\n");
49
50     // We could use gpio_set_dir_out_masked() here
51     for (int gpio = FIRST_GPIO; gpio < FIRST_GPIO + 7; gpio++) {

```

```

52     gpio_init(gpio);
53     gpio_set_dir(gpio, GPIO_OUT);
54     // Our bitmap above has a bit set where we need an LED on, BUT, we are pulling low to
    light
55     // so invert our output
56     gpio_set_outover(gpio, GPIO_OVERRIDE_INVERT);
57 }
58
59 gpio_init(BUTTON_GPIO);
60 gpio_set_dir(BUTTON_GPIO, GPIO_IN);
61 // We are using the button to pull down to 0v when pressed, so ensure that when
62 // unpressed, it uses internal pull ups. Otherwise when unpressed, the input will
63 // be floating.
64 gpio_pull_up(BUTTON_GPIO);
65
66 int val = 0;
67 while (true) {
68     // Count upwards or downwards depending on button input
69     // We are pulling down on switch active, so invert the get to make
70     // a press count downwards
71     if (!gpio_get(BUTTON_GPIO)) {
72         if (val == 9) {
73             val = 0;
74         } else {
75             val++;
76         }
77     } else if (val == 0) {
78         val = 9;
79     } else {
80         val--;
81     }
82
83     // We are starting with GPIO 2, our bitmap starts at bit 0 so shift to start at 2.
84     int32_t mask = bits[val] << FIRST_GPIO;
85
86     // Set all our GPIOs in one go!
87     // If something else is using GPIO, we might want to use gpio_put_masked()
88     gpio_set_mask(mask);
89     sleep_ms(250);
90     gpio_clr_mask(mask);
91 }
92 }
93 /// \end::hello_gpio[]

```

Bill of Materials

Table 36. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
7 segment LED module	1	generic part
68 ohm resistor	7	generic part
DIL push to make switch	1	generic switch
M/M Jumper wires	10	generic part

DHT-11, DHT-22, and AM2302 Sensors

The DHT sensors are fairly well known hobbyist sensors for measuring relative humidity and temperature using a capacitive humidity sensor, and a thermistor. While they are slow, one reading every ~2 seconds, they are reliable and good for basic data logging. Communication is based on a custom protocol which uses a single wire for data.

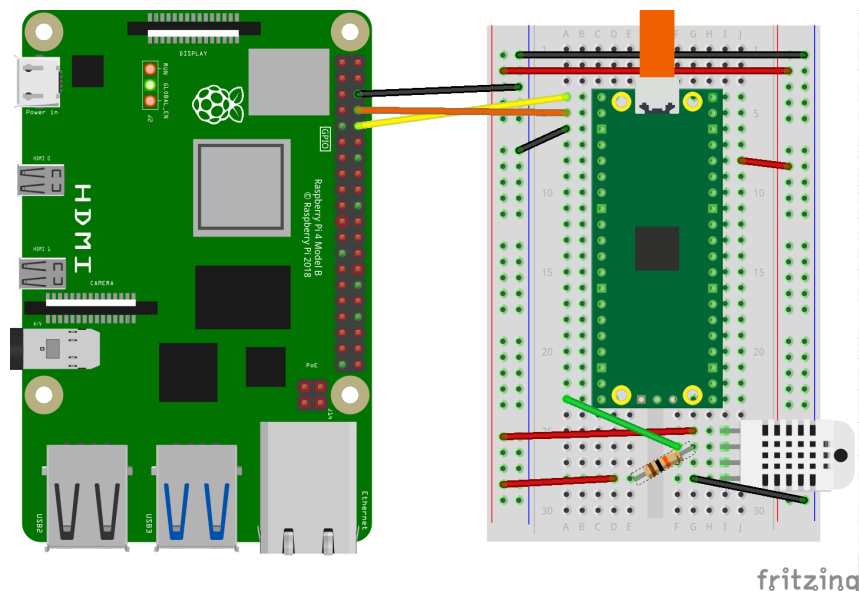
NOTE

The DHT-11 and DHT-22 sensors are the most common. They use the same protocol but have different characteristics, the DHT-22 has better accuracy, and has a larger sensor range than the DHT-11. The sensor is available from a number of retailers.

Wiring information

See [Figure 10](#) for wiring instructions.

Figure 10. Wiring the DHT-22 temperature sensor to Raspberry Pi Pico, and connecting Pico's UART0 to the Raspberry Pi 4.



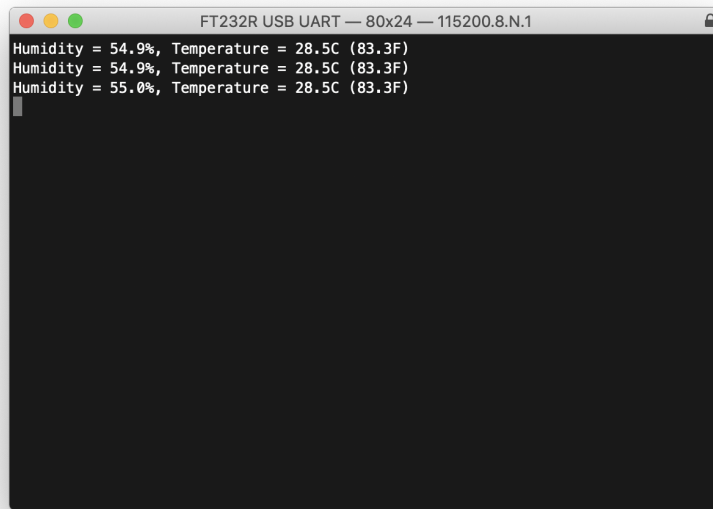
NOTE

One of the pins (pin 3) on the DHT sensor will not be connected, it is not used.

You will want to place a 10 kΩ resistor between VCC and the data pin, to act as a medium-strength pull up on the data line.

Connecting UART0 of Pico to Raspberry Pi as in [Figure 10](#) and you should see something similar to [Figure 11](#) in [minicom](#) when connected to `/dev/serial0` on the Raspberry Pi.

Figure 11. Serial output over Pico's UART0 in a terminal window.



Connect to `/dev/serial0` by typing,

```
$ minicom -b 115200 -o -D /dev/serial0
```

at the command line.

List of Files

A list of files with descriptions of their function;

CMakeLists.txt

Make file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/gpio/dht_sensor/CMakeLists.txt

```
1 add_executable(dht
2     dht.c
3 )
4
5 target_link_libraries(dht pico_stdlib)
6
7 pico_add_extra_outputs(dht)
8
9 # add url via pico_set_program_url
10 example_auto_set_url(dht)
```

dht.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/gpio/dht_sensor/dht.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
```

```

4  * SPDX-License-Identifier: BSD-3-Clause
5  **/
6
7  #include <stdio.h>
8  #include <math.h>
9  #include "pico/stdlib.h"
10 #include "hardware/gpio.h"
11
12 #ifdef PICO_DEFAULT_LED_PIN
13 #define LED_PIN PICO_DEFAULT_LED_PIN
14 #endif
15
16 const uint DHT_PIN = 15;
17 const uint MAX_TIMINGS = 85;
18
19 typedef struct {
20     float humidity;
21     float temp_celsius;
22 } dht_reading;
23
24 void read_from_dht(dht_reading *result);
25
26 int main() {
27     stdio_init_all();
28     gpio_init(DHT_PIN);
29 #ifdef LED_PIN
30     gpio_init(LED_PIN);
31     gpio_set_dir(LED_PIN, GPIO_OUT);
32 #endif
33     while (1) {
34         dht_reading reading;
35         read_from_dht(&reading);
36         float fahrenheit = (reading.temp_celsius * 9 / 5) + 32;
37         printf("Humidity = %.1f%%, Temperature = %.1fC (%.1fF)\n",
38             reading.humidity, reading.temp_celsius, fahrenheit);
39
40         sleep_ms(2000);
41     }
42 }
43
44 void read_from_dht(dht_reading *result) {
45     int data[5] = {0, 0, 0, 0, 0};
46     uint last = 1;
47     uint j = 0;
48
49     gpio_set_dir(DHT_PIN, GPIO_OUT);
50     gpio_put(DHT_PIN, 0);
51     sleep_ms(20);
52     gpio_set_dir(DHT_PIN, GPIO_IN);
53
54 #ifdef LED_PIN
55     gpio_put(LED_PIN, 1);
56 #endif
57     for (uint i = 0; i < MAX_TIMINGS; i++) {
58         uint count = 0;
59         while (gpio_get(DHT_PIN) == last) {
60             count++;
61             sleep_us(1);
62             if (count == 255) break;
63         }
64         last = gpio_get(DHT_PIN);
65         if (count == 255) break;
66
67         if ((i >= 4) && (i % 2 == 0)) {

```

```
68         data[j / 8] <= 1;
69         if (count > 16) data[j / 8] |= 1;
70         j++;
71     }
72 }
73 #ifdef LED_PIN
74     gpio_put(LED_PIN, 0);
75 #endif
76
77 if ((j >= 40) && (data[4] == ((data[0] + data[1] + data[2] + data[3]) & 0xFF))) {
78     result->humidity = (float) ((data[0] << 8) + data[1]) / 10;
79     if (result->humidity > 100) {
80         result->humidity = data[0];
81     }
82     result->temp_celsius = (float) (((data[2] & 0x7F) << 8) + data[3]) / 10;
83     if (result->temp_celsius > 125) {
84         result->temp_celsius = data[2];
85     }
86     if (data[2] & 0x80) {
87         result->temp_celsius = -result->temp_celsius;
88     }
89 } else {
90     printf("Bad data\n");
91 }
92 }
```

Bill of Materials

Table 37. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
10 kΩ resistor	1	generic part
M/M Jumper wires	4	generic part
DHT-22 sensor	1	generic part

Attaching a 16x2 LCD via TTL

This example code shows how to interface the Raspberry Pi Pico to one of the very common 16x2 LCD character displays. Due to the large number of pins these displays use, they are commonly used with extra drivers or backpacks. In this example, we will use an Adafruit LCD display backpack, which supports communication over USB or TTL. A monochrome display with an RGB backlight is also used, but the backpack is compatible with monochrome backlight displays too. There is another example that uses I2C to control a 16x2 display.

The backpack processes a set of commands that are documented [here](#) and preceded by the "special" byte 0xFE. The backpack does the ASCII character conversion and even supports custom character creation. In this example, we use the Pico's primary UART (uart0) to read characters from our computer and send them via the other UART (uart1) to print them onto the LCD. We also define a special startup sequence and vary the display's backlight color.

NOTE

You can change where stdio output goes (Pico's USB, uart0 or both) with CMake directives. The CMakeLists.txt file shows how to enable both.

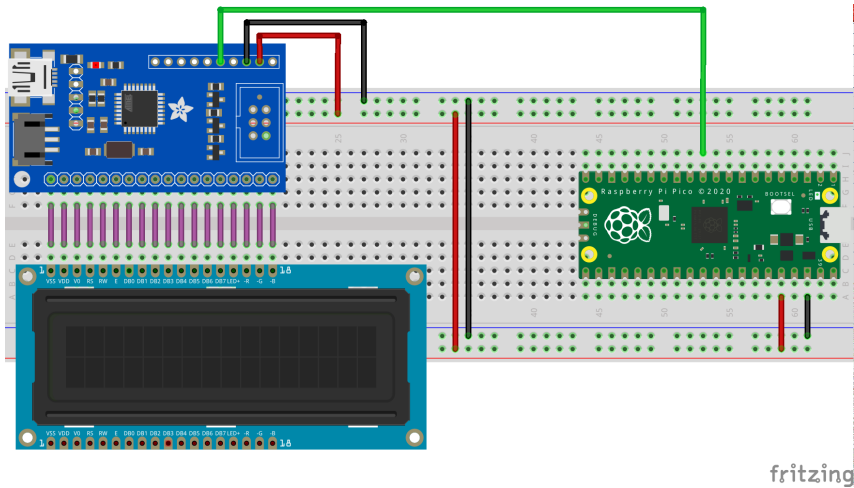
Wiring information

Wiring up the backpack to the Pico requires 3 jumpers, to connect VCC (3.3v), GND, TX. The example here uses both of the Pico's UARTs, one (uart0) for stdio and the other (uart1) for communication with the backpack. Pin 8 is used as the TX pin. Power is supplied from the 3.3V pin. To connect the backpack to the display, it is common practice to solder it onto the back of the display, or during the prototyping stage to use the same parallel lanes on a breadboard.

NOTE

While this display will work at 3.3V, it will be quite dim. Using a 5V source will make it brighter.

Figure 12. Wiring Diagram for LCD with TTL backpack.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/uart/lcd_uart/CMakeLists.txt

```
1 add_executable(lcd_uart
2     lcd_uart.c
3 )
4
5 # pull in common dependencies and additional uart hardware support
6 target_link_libraries(lcd_uart pico_stdlib hardware_uart)
7
8 # enable usb output and uart output
9 # modify here as required
10 pico_enable_stdio_usb(lcd_uart 1)
11 pico_enable_stdio_uart(lcd_uart 1)
12
13 # create map/bin/hex file etc.
14 pico_add_extra_outputs(lcd_uart)
15
16 # add url via pico_set_program_url
```



```
17 example_auto_set_url(lcd_uart)
```

lcd_uart.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/uart/lcd_uart/lcd_uart.c

```
1  /**
2   * Copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3   *
4   * SPDX-License-Identifier: BSD-3-Clause
5   */
6
7  /* Example code to drive a 16x2 LCD panel via an Adafruit TTL LCD "backpack"
8
9   Optionally, the backpack can be connected the VBUS (pin 40) at 5V if
10  the Pico in question is powered by USB for greater brightness.
11
12  If this is done, then no other connections should be made to the backpack apart
13  from those listed below as the backpack's logic levels will change.
14
15  Connections on Raspberry Pi Pico board, other boards may vary.
16
17  GPIO 8 (pin 11)-> RX on backpack
18  3.3v (pin 36) -> 3.3v on backpack
19  GND (pin 38) -> GND on backpack
20 */
21
22 #include <stdio.h>
23 #include <math.h>
24 #include "pico/stdlib.h"
25 #include "pico/binary_info.h"
26 #include "hardware/uart.h"
27
28 // leave uart0 free for stdio
29 #define UART_ID uart1
30 #define BAUD_RATE 9600
31 #define UART_TX_PIN 8
32 #define LCD_WIDTH 16
33 #define LCD_HEIGHT 2
34
35 // basic commands
36 #define LCD_DISPLAY_ON 0x42
37 #define LCD_DISPLAY_OFF 0x46
38 #define LCD_SET_BRIGHTNESS 0x99
39 #define LCD_SET_CONTRAST 0x50
40 #define LCD_AUTOSCROLL_ON 0x51
41 #define LCD_AUTOSCROLL_OFF 0x52
42 #define LCD_CLEAR_SCREEN 0x58
43 #define LCD_SET_SPLASH 0x40
44
45 // cursor commands
46 #define LCD_SET_CURSOR_POS 0x47
47 #define LCD_CURSOR_HOME 0x48
48 #define LCD_CURSOR_BACK 0x4C
49 #define LCD_CURSOR_FORWARD 0x4D
50 #define LCD_UNDERLINE_CURSOR_ON 0x4A
51 #define LCD_UNDERLINE_CURSOR_OFF 0x4B
52 #define LCD_BLOCK_CURSOR_ON 0x53
53 #define LCD_BLOCK_CURSOR_OFF 0x54
54
55 // rgb commands
```

```

56 #define LCD_SET_BACKLIGHT_COLOR 0xD0
57 #define LCD_SET_DISPLAY_SIZE 0xD1
58
59 // change to 0 if display is not RGB capable
60 #define LCD_IS_RGB 1
61
62 void lcd_write(uint8_t cmd, uint8_t* buf, uint8_t buflen) {
63     // all commands are prefixed with 0xFE
64     const uint8_t pre = 0xFE;
65     uart_write_blocking(UART_ID, &pre, 1);
66     uart_write_blocking(UART_ID, &cmd, 1);
67     uart_write_blocking(UART_ID, buf, buflen);
68     sleep_ms(10); // give the display some time
69 }
70
71 void lcd_set_size(uint8_t w, uint8_t h) {
72     // sets the dimensions of the display
73     uint8_t buf[] = { w, h };
74     lcd_write(LCD_SET_DISPLAY_SIZE, buf, 2);
75 }
76
77 void lcd_set_contrast(uint8_t contrast) {
78     // sets the display contrast
79     lcd_write(LCD_SET_CONTRAST, &contrast, 1);
80 }
81
82 void lcd_set_brightness(uint8_t brightness) {
83     // sets the backlight brightness
84     lcd_write(LCD_SET_BRIGHTNESS, &brightness, 1);
85 }
86
87 void lcd_set_cursor(bool is_on) {
88     // set is_on to true if we want the blinking block and underline cursor to show
89     if (is_on) {
90         lcd_write(LCD_BLOCK_CURSOR_ON, NULL, 0);
91         lcd_write(LCD_UNDERLINE_CURSOR_ON, NULL, 0);
92     } else {
93         lcd_write(LCD_BLOCK_CURSOR_OFF, NULL, 0);
94         lcd_write(LCD_UNDERLINE_CURSOR_OFF, NULL, 0);
95     }
96 }
97
98 void lcd_set_backlight(bool is_on) {
99     // turn the backlight on (true) or off (false)
100     if (is_on) {
101         lcd_write(LCD_DISPLAY_ON, (uint8_t *) 0, 1);
102     } else {
103         lcd_write(LCD_DISPLAY_OFF, NULL, 0);
104     }
105 }
106
107 void lcd_clear() {
108     // clear the contents of the display
109     lcd_write(LCD_CLEAR_SCREEN, NULL, 0);
110 }
111
112 void lcd_cursor_reset() {
113     // reset the cursor to (1, 1)
114     lcd_write(LCD_CURSOR_HOME, NULL, 0);
115 }
116
117 #if LCD_IS_RGB
118 void lcd_set_backlight_color(uint8_t r, uint8_t g, uint8_t b) {
119     // only supported on RGB displays!

```

```

120     uint8_t buf[] = { r, g, b };
121     lcd_write(LCD_SET_BACKLIGHT_COLOR, buf, 3);
122 }
123 #endif
124
125 void lcd_init() {
126     lcd_set_backlight(true);
127     lcd_set_size(LCD_WIDTH, LCD_HEIGHT);
128     lcd_set_contrast(155);
129     lcd_set_brightness(255);
130     lcd_set_cursor(false);
131 }
132
133 int main() {
134     stdio_init_all();
135     uart_init(UART_ID, BAUD_RATE);
136     uart_set_translate_crlf(UART_ID, false);
137     gpio_set_function(UART_TX_PIN, UART_FUNCSEL_NUM(UART_ID, UART_TX_PIN));
138
139     bi_decl(bi_1pin_with_func(UART_TX_PIN, UART_FUNCSEL_NUM(UART_ID, UART_TX_PIN)));
140
141     lcd_init();
142
143     // define startup sequence and save to EEPROM
144     // no more or less than 32 chars, if not enough, fill remaining ones with spaces
145     uint8_t splash_buf[] = "Hello LCD, from Pi Towers! ";
146     lcd_write(LCD_SET_SPLASH, splash_buf, LCD_WIDTH * LCD_HEIGHT);
147
148     lcd_cursor_reset();
149     lcd_clear();
150
151     #if LCD_IS_RGB
152     uint8_t i = 0; // it's ok if this overflows and wraps, we're using sin
153     const float frequency = 0.1f;
154     uint8_t red, green, blue;
155     #endif
156
157     while (1) {
158         // send any chars from stdio straight to the backpack
159         char c = getchar();
160         // any bytes not followed by 0xFE (the special command) are interpreted
161         // as text to be displayed on the backpack, so we just send the char
162         // down the UART byte pipe!
163         if (c < 128) uart_putc_raw(UART_ID, c); // skip extra non-ASCII chars
164         #if LCD_IS_RGB
165         // change the display color on keypress, rainbow style!
166         red = (uint8_t)(sin(frequency * i + 0) * 127 + 128);
167         green = (uint8_t)(sin(frequency * i + 2) * 127 + 128);
168         blue = (uint8_t)(sin(frequency * i + 4) * 127 + 128);
169         lcd_set_backlight_color(red, green, blue);
170         i++;
171         #endif
172     }
173 }

```

Bill of Materials

Table 38. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part

Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
16x2 RGB LCD panel 3.3v	1	generic part, available on Adafruit
16x2 LCD backpack	1	from Adafruit
M/M Jumper wires	3	generic part

Attaching a microphone using the ADC

This example code shows how to interface the Raspberry Pi Pico with a standard analog microphone via the onboard analog to digital converter (ADC). In this example, we use an ICS-40180 breakout board by SparkFun but any analog microphone should be compatible with this tutorial. SparkFun have [written a guide](#) for this board that goes into more detail about the board and how it works.

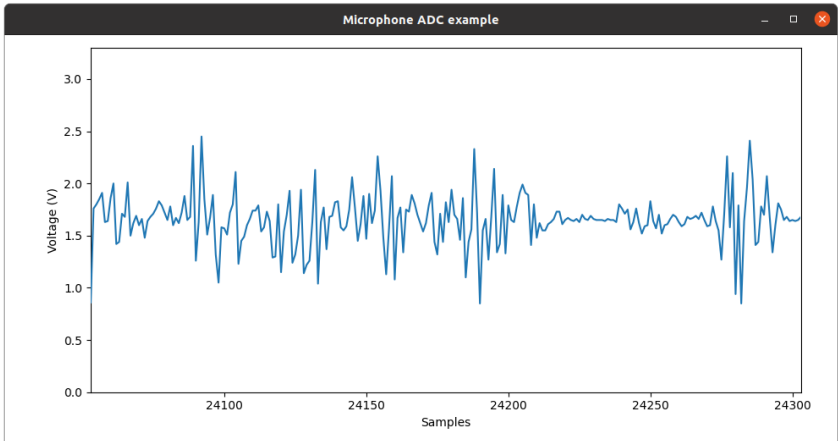
 TIP

An analog to digital converter (ADC) is responsible for reading continually varying input signals that may range from 0 to a specified reference voltage (in the Pico's case this reference voltage is set by the supply voltage and can be measured on pin 35, ADC_VREF) and converting them into binary, i.e. a number that can be digitally stored.

The Pico has a 12-bit ADC (ENOB of 8.7-bit, see [RP2040 datasheet section 4.9.3 for more details](#)), meaning that a read operation will return a number ranging from 0 to 4095 ($2^{12} - 1$) for a total of 4096 possible values. Therefore, the resolution of the ADC is $3.3/4096$, so roughly steps of 0.8 millivolts. The SparkFun breakout uses an OPA344 operational amplifier to boost the signal coming from the microphone to voltage levels that can be easily read by the ADC. An important side effect is that a bias of $0.5 \cdot V_{cc}$ is added to the signal, even when the microphone is not picking up any sound.

The ADC provides us with a raw voltage value but when dealing with sound, we're more interested in the amplitude of the audio signal. This is defined as one half the peak-to-peak amplitude. Included with this example is a very simple Python script that will plot the voltage values it receives via the serial port. By tweaking the sampling rates, and various other parameters, the data from the microphone can be analysed in various ways, such as in a Fast Fourier Transform to see what frequencies make up the signal.

Figure 13. Example output from included Python script



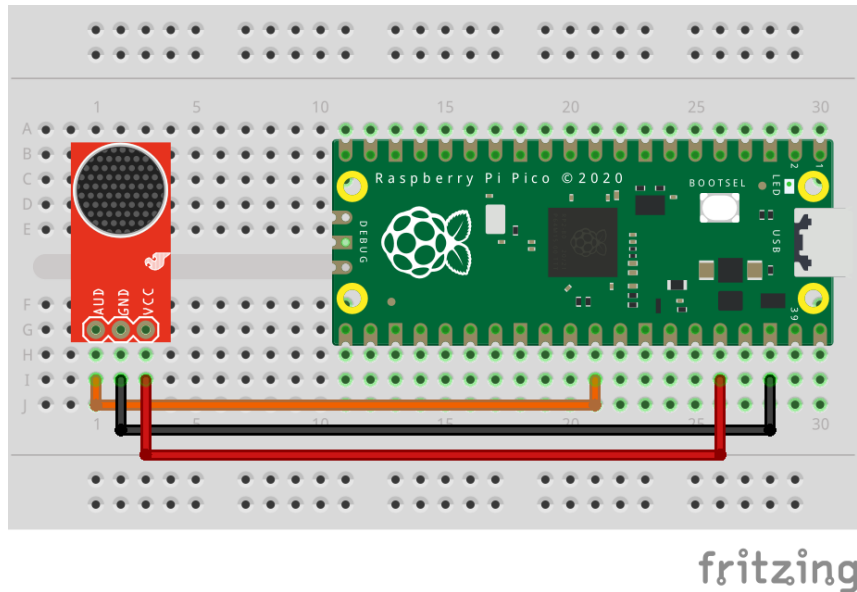
Wiring information

Wiring up the device requires 3 jumpers, to connect VCC (3.3v), GND, and AOUT. The example here uses ADC0, which is GP26. Power is supplied from the 3.3V pin.

WARNING

Most boards will take a range of VCC voltages from the Pico's default 3.3V to the 5 volts commonly seen on other microcontrollers. Ensure your board doesn't output an analogue signal greater than 3.3V as this may result in permanent damage to the Pico's ADC.

Figure 14. Wiring Diagram for ICS-40180 microphone breakout board.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/adc/microphone_adc/CMakeLists.txt

```
1 add_executable(microphone_adc
2     microphone_adc.c
3 )
4
5 # pull in common dependencies and adc hardware support
6 target_link_libraries(microphone_adc pico_stdlib hardware_adc)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(microphone_adc)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(microphone_adc)
```

microphone_adc.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/adc/microphone_adc/microphone_adc.c

```
1 /**
2  * Copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
```

```

7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "hardware/gpio.h"
10 #include "hardware/adc.h"
11 #include "hardware/uart.h"
12 #include "pico/binary_info.h"
13
14 /* Example code to extract analog values from a microphone using the ADC
15    with accompanying Python file to plot these values
16
17    Connections on Raspberry Pi Pico board, other boards may vary.
18
19    GPIO 26/ADC0 (pin 31)-> AOUT or AUD on microphone board
20    3.3v (pin 36) -> VCC on microphone board
21    GND (pin 38) -> GND on microphone board
22 */
23
24 #define ADC_NUM 0
25 #define ADC_PIN (26 + ADC_NUM)
26 #define ADC_VREF 3.3
27 #define ADC_RANGE (1 << 12)
28 #define ADC_CONVERT (ADC_VREF / (ADC_RANGE - 1))
29
30 int main() {
31     stdio_init_all();
32     printf("Beep boop, listening...\n");
33
34     bi_decl(bi_program_description("Analog microphone example for Raspberry Pi Pico")); //
35     for picotool
36     bi_decl(bi_1pin_with_name(ADC_PIN, "ADC input pin"));
37
38     adc_init();
39     adc_gpio_init( ADC_PIN);
40     adc_select_input( ADC_NUM);
41
42     uint adc_raw;
43     while (1) {
44         adc_raw = adc_read(); // raw voltage from ADC
45         printf("%.2f\n", adc_raw * ADC_CONVERT);
46         sleep_ms(10);
47     }

```

Bill of Materials

Table 39. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
ICS-40180 microphone breakout board or similar	1	From SparkFun
M/M Jumper wires	3	generic part

Attaching a BME280 temperature/humidity/pressure sensor via SPI

This example code shows how to interface the Raspberry Pi Pico to a BME280 temperature/humidity/pressure. The particular device used can be interfaced via I2C or SPI, we are using SPI, and interfacing at 3.3v.

This examples reads the data from the sensor, and runs it through the appropriate compensation routines (see the chip datasheet for details <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>). At startup the compensation parameters required by the compensation routines are read from the chip.)

Wiring information

Wiring up the device requires 6 jumpers as follows:

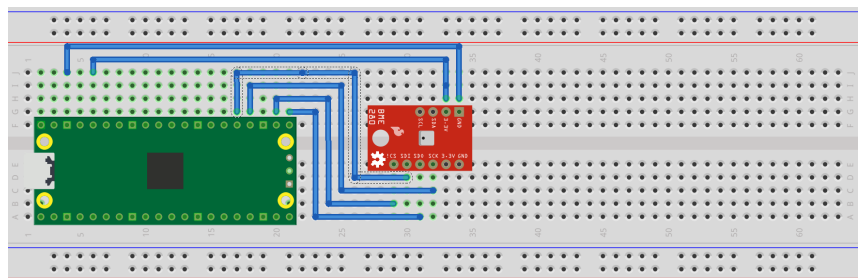
- GPIO 16 (pin 21) MISO/spi0_rx → SDO/SDO on bme280 board
- GPIO 17 (pin 22) Chip select → CSB/!CS on bme280 board
- GPIO 18 (pin 24) SCK/spi0_sclk → SCL/SCK on bme280 board
- GPIO 19 (pin 25) MOSI/spi0_tx → SDA/SDI on bme280 board
- 3.3v (pin 3;6) → VCC on bme280 board
- GND (pin 38) → GND on bme280 board

The example here uses SPI port 0. Power is supplied from the 3.3V pin.

NOTE

There are many different manufacturers who sell boards with the BME280. Whilst they all appear slightly different, they all have, at least, the same 6 pins required to power and communicate. When wiring up a board that is different to the one in the diagram, ensure you connect up as described in the previous paragraph.

Figure 15. Wiring Diagram for bme280.



fritzing

List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/bme280_spi/CMakeLists.txt

```
1 add_executable(bme280_spi
2     bme280_spi.c
3 )
4
5 # pull in common dependencies and additional spi hardware support
6 target_link_libraries(bme280_spi pico_stdlib hardware_spi)
```

```

7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(bme280_spi)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(bme280_spi)

```

bme280_spi.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/bme280_spi/bme280_spi.c

```

1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/spi.h"
12
13 /* Example code to talk to a bme280 humidity/temperature/pressure sensor.
14
15  NOTE: Ensure the device is capable of being driven at 3.3v NOT 5v. The Pico
16  GPIO (and therefore SPI) cannot be used at 5v.
17
18  You will need to use a level shifter on the SPI lines if you want to run the
19  board at 5v.
20
21  Connections on Raspberry Pi Pico board and a generic bme280 board, other
22  boards may vary.
23
24  GPIO 16 (pin 21) MISO/spi0_rx-> SDO/SD0 on bme280 board
25  GPIO 17 (pin 22) Chip select -> CSB/!CS on bme280 board
26  GPIO 18 (pin 24) SCK/spi0_sclk -> SCL/SCK on bme280 board
27  GPIO 19 (pin 25) MOSI/spi0_tx -> SDA/SDI on bme280 board
28  3.3v (pin 36) -> VCC on bme280 board
29  GND (pin 38) -> GND on bme280 board
30
31  Note: SPI devices can have a number of different naming schemes for pins. See
32  the Wikipedia page at https://en.wikipedia.org/wiki/Serial\_Peripheral\_Interface
33  for variations.
34
35  This code uses a bunch of register definitions, and some compensation code derived
36  from the Bosch datasheet which can be found here.
37  https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-
38  ds002.pdf
39  */
40 #define READ_BIT 0x80
41
42 int32_t t_fine;
43
44 uint16_t dig_T1;
45 int16_t dig_T2, dig_T3;
46 uint16_t dig_P1;
47 int16_t dig_P2, dig_P3, dig_P4, dig_P5, dig_P6, dig_P7, dig_P8, dig_P9;
48 uint8_t dig_H1, dig_H3;
49 int8_t dig_H6;

```



```

50 int16_t dig_H2, dig_H4, dig_H5;
51
52 /* The following compensation functions are required to convert from the raw ADC
53 data from the chip to something usable. Each chip has a different set of
54 compensation parameters stored on the chip at point of manufacture, which are
55 read from the chip at startup and used in these routines.
56 */
57 int32_t compensate_temp(int32_t adc_T) {
58     int32_t var1, var2, T;
59     var1 = (((adc_T >> 3) - ((int32_t) dig_T1 << 1))) * ((int32_t) dig_T2) >> 11;
60     var2 = (((((adc_T >> 4) - ((int32_t) dig_T1)) * ((adc_T >> 4) - ((int32_t) dig_T1))) >>
12) * ((int32_t) dig_T3))
61         >> 14;
62
63     t_fine = var1 + var2;
64     T = (t_fine * 5 + 128) >> 8;
65     return T;
66 }
67
68 uint32_t compensate_pressure(int32_t adc_P) {
69     int32_t var1, var2;
70     uint32_t p;
71     var1 = (((int32_t) t_fine) >> 1) - (int32_t) 64000;
72     var2 = (((var1 >> 2) * (var1 >> 2)) >> 11) * ((int32_t) dig_P6);
73     var2 = var2 + ((var1 * ((int32_t) dig_P5)) << 1);
74     var2 = (var2 >> 2) + (((int32_t) dig_P4) << 16);
75     var1 = (((dig_P3 * (((var1 >> 2) * (var1 >> 2)) >> 13)) >> 3) + (((int32_t) dig_P2) *
var1) >> 1)) >> 18;
76     var1 = (((32768 + var1)) * ((int32_t) dig_P1)) >> 15;
77     if (var1 == 0)
78         return 0;
79
80     p = (((uint32_t) (((int32_t) 1048576) - adc_P) - (var2 >> 12))) * 3125;
81     if (p < 0x80000000)
82         p = (p << 1) / ((uint32_t) var1);
83     else
84         p = (p / (uint32_t) var1) * 2;
85
86     var1 = (((int32_t) dig_P9) * ((int32_t) (((p >> 3) * (p >> 3)) >> 13))) >> 12;
87     var2 = (((int32_t) (p >> 2)) * ((int32_t) dig_P8)) >> 13;
88     p = (uint32_t) ((int32_t) p + ((var1 + var2 + dig_P7) >> 4));
89
90     return p;
91 }
92
93 uint32_t compensate_humidity(int32_t adc_H) {
94     int32_t v_x1_u32r;
95     v_x1_u32r = (t_fine - ((int32_t) 76800));
96     v_x1_u32r = (((((adc_H << 14) - (((int32_t) dig_H4) << 20) - (((int32_t) dig_H5) *
v_x1_u32r)) +
97         ((int32_t) 16384)) >> 15) * (((((((v_x1_u32r * ((int32_t) dig_H6)) >> 10)
* ((v_x1_u32r *
98         ((int32_t) dig_H3))
99         >> 11) + ((int32_t) 32768))) >> 10) + ((int32_t) 2097152)) *
100         ((int32_t) dig_H2) + 8192) >> 14));
101     v_x1_u32r = (v_x1_u32r - (((((v_x1_u32r >> 15) * (v_x1_u32r >> 15)) >> 7) * ((int32_t)
dig_H1)) >> 4));
102     v_x1_u32r = (v_x1_u32r < 0 ? 0 : v_x1_u32r);
103     v_x1_u32r = (v_x1_u32r > 419430400 ? 419430400 : v_x1_u32r);
104
105     return (uint32_t) (v_x1_u32r >> 12);
106 }
107

```

```

108 #ifdef PICO_DEFAULT_SPI_CSN_PIN
109 static inline void cs_select() {
110     asm volatile("nop \n nop \n nop");
111     gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 0); // Active low
112     asm volatile("nop \n nop \n nop");
113 }
114
115 static inline void cs_deselect() {
116     asm volatile("nop \n nop \n nop");
117     gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 1);
118     asm volatile("nop \n nop \n nop");
119 }
120 #endif
121
122 #if defined(spi_default) && defined(PICO_DEFAULT_SPI_CSN_PIN)
123 static void write_register(uint8_t reg, uint8_t data) {
124     uint8_t buf[2];
125     buf[0] = reg & 0x7f; // remove read bit as this is a write
126     buf[1] = data;
127     cs_select();
128     spi_write_blocking(spi_default, buf, 2);
129     cs_deselect();
130     sleep_ms(10);
131 }
132
133 static void read_registers(uint8_t reg, uint8_t *buf, uint16_t len) {
134     // For this particular device, we send the device the register we want to read
135     // first, then subsequently read from the device. The register is auto incrementing
136     // so we don't need to keep sending the register we want, just the first.
137     reg |= READ_BIT;
138     cs_select();
139     spi_write_blocking(spi_default, &reg, 1);
140     sleep_ms(10);
141     spi_read_blocking(spi_default, 0, buf, len);
142     cs_deselect();
143     sleep_ms(10);
144 }
145
146 /* This function reads the manufacturing assigned compensation parameters from the device */
147 void read_compensation_parameters() {
148     uint8_t buffer[26];
149
150     read_registers(0x88, buffer, 26);
151
152     dig_T1 = buffer[0] | (buffer[1] << 8);
153     dig_T2 = buffer[2] | (buffer[3] << 8);
154     dig_T3 = buffer[4] | (buffer[5] << 8);
155
156     dig_P1 = buffer[6] | (buffer[7] << 8);
157     dig_P2 = buffer[8] | (buffer[9] << 8);
158     dig_P3 = buffer[10] | (buffer[11] << 8);
159     dig_P4 = buffer[12] | (buffer[13] << 8);
160     dig_P5 = buffer[14] | (buffer[15] << 8);
161     dig_P6 = buffer[16] | (buffer[17] << 8);
162     dig_P7 = buffer[18] | (buffer[19] << 8);
163     dig_P8 = buffer[20] | (buffer[21] << 8);
164     dig_P9 = buffer[22] | (buffer[23] << 8);
165
166     dig_H1 = buffer[25]; // 0xA1
167
168     read_registers(0xE1, buffer, 8);
169
170     dig_H2 = buffer[0] | (buffer[1] << 8); // 0xE1 | 0xE2
171     dig_H3 = (int8_t) buffer[2]; // 0xE3

```

```

172     dig_H4 = buffer[3] << 4 | (buffer[4] & 0xf); // 0xE4 | 0xE5[3:0]
173     dig_H5 = (buffer[4] >> 4) | (buffer[5] << 4); // 0xE5[7:4] | 0xE6
174     dig_H6 = (int8_t) buffer[6]; // 0xE7
175 }
176
177 static void bme280_read_raw(int32_t *humidity, int32_t *pressure, int32_t *temperature) {
178     uint8_t buffer[8];
179
180     read_registers(0xF7, buffer, 8);
181     *pressure = ((uint32_t) buffer[0] << 12) | ((uint32_t) buffer[1] << 4) | (buffer[2] >>
182         4);
183     *temperature = ((uint32_t) buffer[3] << 12) | ((uint32_t) buffer[4] << 4) | (buffer[5]
184         >> 4);
185     *humidity = (uint32_t) buffer[6] << 8 | buffer[7];
186 }
187 #endif
188
189 int main() {
190     stdio_init_all();
191     #if !defined(spi_default) || !defined(PICO_DEFAULT_SPI_SCK_PIN) ||
192         !defined(PICO_DEFAULT_SPI_TX_PIN) || !defined(PICO_DEFAULT_SPI_RX_PIN) ||
193         !defined(PICO_DEFAULT_SPI_CSN_PIN)
194     #warning spi/bme280_spi example requires a board with SPI pins
195     puts("Default SPI pins were not defined");
196 #else
197     printf("Hello, bme280! Reading raw data from registers via SPI...\n");
198
199     // This example will use SPI0 at 0.5MHz.
200     spi_init(spi_default, 500 * 1000);
201     gpio_set_function(PICO_DEFAULT_SPI_RX_PIN, GPIO_FUNC_SPI);
202     gpio_set_function(PICO_DEFAULT_SPI_SCK_PIN, GPIO_FUNC_SPI);
203     gpio_set_function(PICO_DEFAULT_SPI_TX_PIN, GPIO_FUNC_SPI);
204     // Make the SPI pins available to picotool
205     bi_decl(bi_3pins_with_func(PICO_DEFAULT_SPI_RX_PIN, PICO_DEFAULT_SPI_TX_PIN,
206         PICO_DEFAULT_SPI_SCK_PIN, GPIO_FUNC_SPI));
207
208     // Chip select is active-low, so we'll initialise it to a driven-high state
209     gpio_init(PICO_DEFAULT_SPI_CSN_PIN);
210     gpio_set_dir(PICO_DEFAULT_SPI_CSN_PIN, GPIO_OUT);
211     gpio_put(PICO_DEFAULT_SPI_CSN_PIN, 1);
212     // Make the CS pin available to picotool
213     bi_decl(bi_1pin_with_name(PICO_DEFAULT_SPI_CSN_PIN, "SPI CS"));
214
215     // See if SPI is working - interrogate the device for its I2C ID number, should be 0x60
216     uint8_t id;
217     read_registers(0xD0, &id, 1);
218     printf("Chip ID is 0x%x\n", id);
219
220     read_compensation_parameters();
221
222     write_register(0xF2, 0x1); // Humidity oversampling register - going for x1
223     write_register(0xF4, 0x27); // Set rest of oversampling modes and run mode to normal
224
225     int32_t humidity, pressure, temperature;
226
227     while (1) {
228         bme280_read_raw(&humidity, &pressure, &temperature);
229
230         // These are the raw numbers from the chip, so we need to run through the
231         // compensations to get human understandable numbers
232         temperature = compensate_temp(temperature);
233         pressure = compensate_pressure(pressure);
234         humidity = compensate_humidity(humidity);

```

```
231
232     printf("Humidity = %.2f%%\n", humidity / 1024.0);
233     printf("Pressure = %dPa\n", pressure);
234     printf("Temp. = %.2fC\n", temperature / 100.0);
235
236     sleep_ms(1000);
237 }
238 #endif
239 }
```

Bill of Materials

Table 40. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
BME280 board	1	generic part
M/M Jumper wires	6	generic part

Attaching a MPU9250 accelerometer/gyroscope via SPI

This example code shows how to interface the Raspberry Pi Pico to the MPU9250 accelerometer/gyroscope board. The particular device used can be interfaced via I2C or SPI, we are using SPI, and interfacing at 3.3v.

NOTE

This is a very basic example, and only recovers raw data from the sensor. There are various calibration options available that should be used to ensure that the final results are accurate. It is also possible to wire up the interrupt pin to a GPIO and read data only when it is ready, rather than using the polling approach in the example.

Wiring information

Wiring up the device requires 6 jumpers as follows:

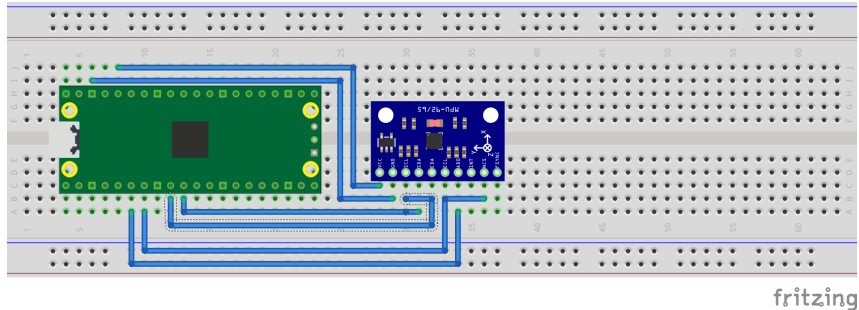
- GPIO 4 (pin 6) MISO/spi0_rx → ADO on MPU9250 board
- GPIO 5 (pin 7) Chip select → NCS on MPU9250 board
- GPIO 6 (pin 9) SCK/spi0_sclk → SCL on MPU9250 board
- GPIO 7 (pin 10) MOSI/spi0_tx → SDA on MPU9250 board
- 3.3v (pin 36) → VCC on MPU9250 board
- GND (pin 38) → GND on MPU9250 board

The example here uses SPI port 0. Power is supplied from the 3.3V pin.

NOTE

There are many different manufacturers who sell boards with the MPU9250. Whilst they all appear slightly different, they all have, at least, the same 6 pins required to power and communicate. When wiring up a board that is different to the one in the diagram, ensure you connect up as described in the previous paragraph.

Figure 16. Wiring Diagram for MPU9250.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/mpu9250_spi/CMakeLists.txt

```
1 add_executable(mpu9250_spi
2     mpu9250_spi.c
3 )
4
5 # pull in common dependencies and additional spi hardware support
6 target_link_libraries(mpu9250_spi pico_stdlib hardware_spi)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(mpu9250_spi)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(mpu9250_spi)
```

mpu9250_spi.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/mpu9250_spi/mpu9250_spi.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/spi.h"
12
13 /* Example code to talk to a MPU9250 MEMS accelerometer and gyroscope.
14    Ignores the magnetometer, that is left as a exercise for the reader.
15 */
```

```

16  This is taking to simple approach of simply reading registers. It's perfectly
17  possible to link up an interrupt line and set things up to read from the
18  inbuilt FIFO to make it more useful.
19
20  NOTE: Ensure the device is capable of being driven at 3.3v NOT 5v. The Pico
21  GPIO (and therefore SPI) cannot be used at 5v.
22
23  You will need to use a level shifter on the I2C lines if you want to run the
24  board at 5v.
25
26  Connections on Raspberry Pi Pico board and a generic MPU9250 board, other
27  boards may vary.
28
29  GPIO 4 (pin 6) MISO/spi0_rx-> ADO on MPU9250 board
30  GPIO 5 (pin 7) Chip select -> NCS on MPU9250 board
31  GPIO 6 (pin 9) SCK/spi0_sclk -> SCL on MPU9250 board
32  GPIO 7 (pin 10) MOSI/spi0_tx -> SDA on MPU9250 board
33  3.3v (pin 36) -> VCC on MPU9250 board
34  GND (pin 38) -> GND on MPU9250 board
35
36  Note: SPI devices can have a number of different naming schemes for pins. See
37  the Wikipedia page at https://en.wikipedia.org/wiki/Serial\_Peripheral\_Interface
38  for variations.
39  The particular device used here uses the same pins for I2C and SPI, hence the
40  using of I2C names
41  */
42
43  #define PIN_MISO 4
44  #define PIN_CS 5
45  #define PIN_SCK 6
46  #define PIN_MOSI 7
47
48  #define SPI_PORT spi0
49  #define READ_BIT 0x80
50
51  static inline void cs_select() {
52      asm volatile("nop \n nop \n nop");
53      gpio_put(PIN_CS, 0); // Active low
54      asm volatile("nop \n nop \n nop");
55  }
56
57  static inline void cs_deselect() {
58      asm volatile("nop \n nop \n nop");
59      gpio_put(PIN_CS, 1);
60      asm volatile("nop \n nop \n nop");
61  }
62
63  static void mpu9250_reset() {
64      // Two byte reset. First byte register, second byte data
65      // There are a load more options to set up the device in different ways that could be
66      // added here
67      uint8_t buf[] = {0x6B, 0x00};
68      cs_select();
69      spi_write_blocking(SPI_PORT, buf, 2);
70      cs_deselect();
71  }
72
73  static void read_registers(uint8_t reg, uint8_t *buf, uint16_t len) {
74      // For this particular device, we send the device the register we want to read
75      // first, then subsequently read from the device. The register is auto incrementing
76      // so we don't need to keep sending the register we want, just the first.
77
78      reg |= READ_BIT;

```

```

79     cs_select();
80     spi_write_blocking(SPI_PORT, &reg, 1);
81     sleep_ms(10);
82     spi_read_blocking(SPI_PORT, 0, buf, len);
83     cs_deselect();
84     sleep_ms(10);
85 }
86
87
88 static void mpu9250_read_raw(int16_t accel[3], int16_t gyro[3], int16_t *temp) {
89     uint8_t buffer[6];
90
91     // Start reading acceleration registers from register 0x3B for 6 bytes
92     read_registers(0x3B, buffer, 6);
93
94     for (int i = 0; i < 3; i++) {
95         accel[i] = (buffer[i * 2] << 8 | buffer[(i * 2) + 1]);
96     }
97
98     // Now gyro data from reg 0x43 for 6 bytes
99     read_registers(0x43, buffer, 6);
100
101     for (int i = 0; i < 3; i++) {
102         gyro[i] = (buffer[i * 2] << 8 | buffer[(i * 2) + 1]);
103     }
104
105     // Now temperature from reg 0x41 for 2 bytes
106     read_registers(0x41, buffer, 2);
107
108     *temp = buffer[0] << 8 | buffer[1];
109 }
110
111 int main() {
112     stdio_init_all();
113
114     printf("Hello, MPU9250! Reading raw data from registers via SPI...\n");
115
116     // This example will use SPI0 at 0.5MHz.
117     spi_init(SPI_PORT, 500 * 1000);
118     gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
119     gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
120     gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
121     // Make the SPI pins available to picotool
122     bi_decl(bi_3pins_with_func(PIN_MISO, PIN_MOSI, PIN_SCK, GPIO_FUNC_SPI));
123
124     // Chip select is active-low, so we'll initialise it to a driven-high state
125     gpio_init(PIN_CS);
126     gpio_set_dir(PIN_CS, GPIO_OUT);
127     gpio_put(PIN_CS, 1);
128     // Make the CS pin available to picotool
129     bi_decl(bi_1pin_with_name(PIN_CS, "SPI CS"));
130
131     mpu9250_reset();
132
133     // See if SPI is working - interrogate the device for its I2C ID number, should be 0x71
134     uint8_t id;
135     read_registers(0x75, &id, 1);
136     printf("I2C address is 0x%x\n", id);
137
138     int16_t acceleration[3], gyro[3], temp;
139
140     while (1) {
141         mpu9250_read_raw(acceleration, gyro, &temp);
142

```

```
143 // These are the raw numbers from the chip, so will need tweaking to be really useful.
144 // See the datasheet for more information
145 printf("Acc. X = %d, Y = %d, Z = %d\n", acceleration[0], acceleration[1],
    acceleration[2]);
146 printf("Gyro. X = %d, Y = %d, Z = %d\n", gyro[0], gyro[1], gyro[2]);
147 // Temperature is simple so use the datasheet calculation to get deg C.
148 // Note this is chip temperature.
149 printf("Temp. = %f\n", (temp / 340.0) + 36.53);
150
151 sleep_ms(100);
152 }
153 }
```

Bill of Materials

Table 41. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
MPU9250 board	1	generic part
M/M Jumper wires	6	generic part

Attaching a MPU6050 accelerometer/gyroscope via I2C

This example code shows how to interface the Raspberry Pi Pico to the MPU6050 accelerometer/gyroscope board. This device uses I2C for communications, and most MPU6050 parts are happy running at either 3.3 or 5v. The Raspberry Pi RP2040 GPIO's work at 3.3v so that is what the example uses.

NOTE

This is a very basic example, and only recovers raw data from the sensor. There are various calibration options available that should be used to ensure that the final results are accurate. It is also possible to wire up the interrupt pin to a GPIO and read data only when it is ready, rather than using the polling approach in the example.

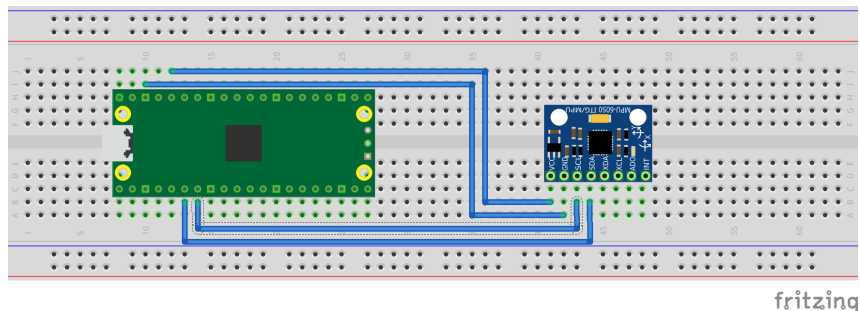
Wiring information

Wiring up the device requires 4 jumpers, to connect VCC (3.3v), GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3.3V pin.

NOTE

There are many different manufacturers who sell boards with the MPU6050. Whilst they all appear slightly different, they all have, at least, the same 4 pins required to power and communicate. When wiring up a board that is different to the one in the diagram, ensure you connect up as described in the previous paragraph.

Figure 17. Wiring
Diagram for MPU6050.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mpu6050_i2c/CMakeLists.txt

```
1 add_executable(mpu6050_i2c
2     mpu6050_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(mpu6050_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(mpu6050_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(mpu6050_i2c)
```

mpu6050_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mpu6050_i2c/mpu6050_i2c.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to a MPU6050 MEMS accelerometer and gyroscope
14
15 This is taking to simple approach of simply reading registers. It's perfectly
16 possible to link up an interrupt line and set things up to read from the
17 inbuilt FIFO to make it more useful.
18
19 NOTE: Ensure the device is capable of being driven at 3.3v NOT 5v. The Pico
20 GPIO (and therefore I2C) cannot be used at 5v.
21
22 You will need to use a level shifter on the I2C lines if you want to run the
```

```

23  board at 5v.
24
25  Connections on Raspberry Pi Pico board, other boards may vary.
26
27  GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is GP4 (pin 6)) -> SDA on MPU6050 board
28  GPIO PICO_DEFAULT_I2C_SCL_PIN (On Pico this is GP5 (pin 7)) -> SCL on MPU6050 board
29  3.3v (pin 36) -> VCC on MPU6050 board
30  GND (pin 38) -> GND on MPU6050 board
31  */
32
33  // By default these devices are on bus address 0x68
34  static int addr = 0x68;
35
36  #ifdef i2c_default
37  static void mpu6050_reset() {
38      // Two byte reset. First byte register, second byte data
39      // There are a load more options to set up the device in different ways that could be
      added here
40      uint8_t buf[] = {0x6B, 0x80};
41      i2c_write_blocking(i2c_default, addr, buf, 2, false);
42      sleep_ms(100); // Allow device to reset and stabilize
43
44      // Clear sleep mode (0x6B register, 0x00 value)
45      buf[1] = 0x00; // Clear sleep mode by writing 0x00 to the 0x6B register
46      i2c_write_blocking(i2c_default, addr, buf, 2, false);
47      sleep_ms(10); // Allow stabilization after waking up
48  }
49
50  static void mpu6050_read_raw(int16_t accel[3], int16_t gyro[3], int16_t *temp) {
51      // For this particular device, we send the device the register we want to read
52      // first, then subsequently read from the device. The register is auto incrementing
53      // so we don't need to keep sending the register we want, just the first.
54
55      uint8_t buffer[6];
56
57      // Start reading acceleration registers from register 0x3B for 6 bytes
58      uint8_t val = 0x3B;
59      i2c_write_blocking(i2c_default, addr, &val, 1, true); // true to keep master control of
      bus
60      i2c_read_blocking(i2c_default, addr, buffer, 6, false);
61
62      for (int i = 0; i < 3; i++) {
63          accel[i] = (buffer[i * 2] << 8 | buffer[(i * 2) + 1]);
64      }
65
66      // Now gyro data from reg 0x43 for 6 bytes
67      // The register is auto incrementing on each read
68      val = 0x43;
69      i2c_write_blocking(i2c_default, addr, &val, 1, true);
70      i2c_read_blocking(i2c_default, addr, buffer, 6, false); // False - finished with bus
71
72      for (int i = 0; i < 3; i++) {
73          gyro[i] = (buffer[i * 2] << 8 | buffer[(i * 2) + 1]);
74      }
75
76      // Now temperature from reg 0x41 for 2 bytes
77      // The register is auto incrementing on each read
78      val = 0x41;
79      i2c_write_blocking(i2c_default, addr, &val, 1, true);
80      i2c_read_blocking(i2c_default, addr, buffer, 2, false); // False - finished with bus
81
82      *temp = buffer[0] << 8 | buffer[1];
83  }
84  #endif

```

```
85
86 int main() {
87     stdio_init_all();
88     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
        !defined(PICO_DEFAULT_I2C_SCL_PIN)
89         #warning i2c/mpu6050_i2c example requires a board with I2C pins
90         puts("Default I2C pins were not defined");
91         return 0;
92     #else
93         printf("Hello, MPU6050! Reading raw data from registers...\n");
94
95         // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
96         i2c_init(i2c_default, 400 * 1000);
97         gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
98         gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
99         gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
100        gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
101        // Make the I2C pins available to picotool
102        bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
        GPIO_FUNC_I2C));
103
104        mpu6050_reset();
105
106        int16_t acceleration[3], gyro[3], temp;
107
108        while (1) {
109            mpu6050_read_raw(acceleration, gyro, &temp);
110
111            // These are the raw numbers from the chip, so will need tweaking to be really useful.
112            // See the datasheet for more information
113            printf("Acc. X = %d, Y = %d, Z = %d\n", acceleration[0], acceleration[1],
        acceleration[2]);
114            printf("Gyro. X = %d, Y = %d, Z = %d\n", gyro[0], gyro[1], gyro[2]);
115            // Temperature is simple so use the datasheet calculation to get deg C.
116            // Note this is chip temperature.
117            printf("Temp. = %f\n", (temp / 340.0) + 36.53);
118
119            sleep_ms(100);
120        }
121    #endif
122 }
```

Bill of Materials

Table 42. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
MPU6050 board	1	generic part
M/M Jumper wires	4	generic part

Attaching a 16x2 LCD via I2C

This example code shows how to interface the Raspberry Pi Pico to one of the very common 16x2 LCD character

displays. The display will need a 3.3V I2C adapter board as this example uses I2C for communications.

i NOTE

These LCD displays can also be driven directly using GPIO without the use of an adapter board. That is beyond the scope of this example.

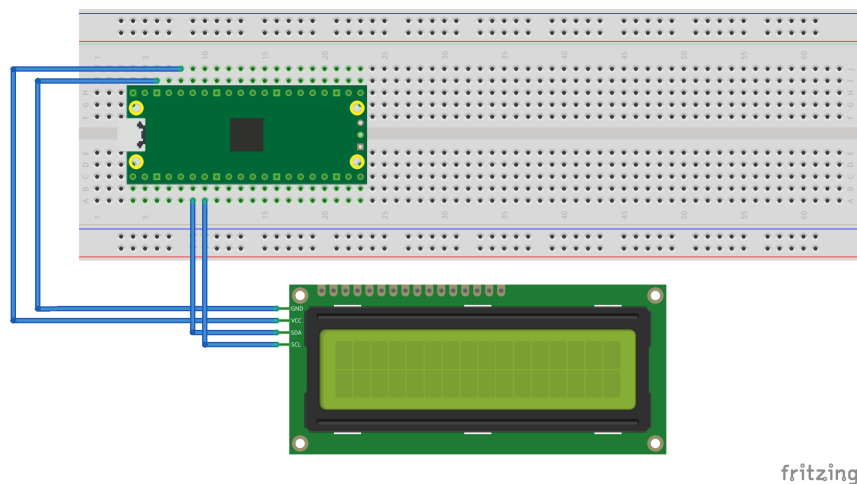
Wiring information

Wiring up the device requires 4 jumpers, to connect VCC (3.3v), GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3.3V pin.

⚠ WARNING

Many displays of this type are 5v. If you wish to use a 5v display you will need to use level shifters on the SDA and SCL lines to convert from the 3.3V used by the RP2040. Whilst a 5v display will just about work at 3.3v, the display will be dim.

Figure 18. Wiring Diagram for LCD1602A LCD with I2C bridge.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/lcd_1602_i2c/CMakeLists.txt

```
1 add_executable(lcd_1602_i2c
2     lcd_1602_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(lcd_1602_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(lcd_1602_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(lcd_1602_i2c)
```

lcd_1602_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/lcd_1602_i2c/lcd_1602_i2c.c

```

1  /**
2   * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3   *
4   * SPDX-License-Identifier: BSD-3-Clause
5   */
6
7  #include <stdio.h>
8  #include <string.h>
9  #include "pico/stdlib.h"
10 #include "hardware/i2c.h"
11 #include "pico/binary_info.h"
12
13 /* Example code to drive a 16x2 LCD panel via a I2C bridge chip (e.g. PCF8574)
14
15     NOTE: The panel must be capable of being driven at 3.3v NOT 5v. The Pico
16     GPIO (and therefore I2C) cannot be used at 5v.
17
18     You will need to use a level shifter on the I2C lines if you want to run the
19     board at 5v.
20
21     Connections on Raspberry Pi Pico board, other boards may vary.
22
23     GPIO 4 (pin 6)-> SDA on LCD bridge board
24     GPIO 5 (pin 7)-> SCL on LCD bridge board
25     3.3v (pin 36) -> VCC on LCD bridge board
26     GND (pin 38) -> GND on LCD bridge board
27 */
28 // commands
29 const int LCD_CLEARDISPLAY = 0x01;
30 const int LCD_RETURNHOME = 0x02;
31 const int LCD_ENTRYMODESET = 0x04;
32 const int LCD_DISPLAYCONTROL = 0x08;
33 const int LCD_CURSORSHIFT = 0x10;
34 const int LCD_FUNCTIONSET = 0x20;
35 const int LCD_SETCGRAMADDR = 0x40;
36 const int LCD_SETDDRAMADDR = 0x80;
37
38 // flags for display entry mode
39 const int LCD_ENTRYSHIFTINCREMENT = 0x01;
40 const int LCD_ENTRYLEFT = 0x02;
41
42 // flags for display and cursor control
43 const int LCD_BLINKON = 0x01;
44 const int LCD_CURSORON = 0x02;
45 const int LCD_DISPLAYON = 0x04;
46
47 // flags for display and cursor shift
48 const int LCD_MOVERIGHT = 0x04;
49 const int LCD_DISPLAYMOVE = 0x08;
50
51 // flags for function set
52 const int LCD_5x10DOTS = 0x04;
53 const int LCD_2LINE = 0x08;
54 const int LCD_8BITMODE = 0x10;
55
56 // flag for backlight control
57 const int LCD_BACKLIGHT = 0x08;
58

```

```

59 const int LCD_ENABLE_BIT = 0x04;
60
61 // By default these LCD display drivers are on bus address 0x27
62 static int addr = 0x27;
63
64 // Modes for lcd_send_byte
65 #define LCD_CHARACTER 1
66 #define LCD_COMMAND 0
67
68 #define MAX_LINES 2
69 #define MAX_CHARS 16
70
71 /* Quick helper function for single byte transfers */
72 void i2c_write_byte(uint8_t val) {
73     #ifdef i2c_default
74         i2c_write_blocking(i2c_default, addr, &val, 1, false);
75     #endif
76 }
77
78 void lcd_toggle_enable(uint8_t val) {
79     // Toggle enable pin on LCD display
80     // We cannot do this too quickly or things don't work
81     #define DELAY_US 600
82     sleep_us(DELAY_US);
83     i2c_write_byte(val | LCD_ENABLE_BIT);
84     sleep_us(DELAY_US);
85     i2c_write_byte(val & ~LCD_ENABLE_BIT);
86     sleep_us(DELAY_US);
87 }
88
89 // The display is sent a byte as two separate nibble transfers
90 void lcd_send_byte(uint8_t val, int mode) {
91     uint8_t high = mode | (val & 0xF0) | LCD_BACKLIGHT;
92     uint8_t low = mode | ((val << 4) & 0xF0) | LCD_BACKLIGHT;
93
94     i2c_write_byte(high);
95     lcd_toggle_enable(high);
96     i2c_write_byte(low);
97     lcd_toggle_enable(low);
98 }
99
100 void lcd_clear(void) {
101     lcd_send_byte(LCD_CLEARDISPLAY, LCD_COMMAND);
102 }
103
104 // go to location on LCD
105 void lcd_set_cursor(int line, int position) {
106     int val = (line == 0) ? 0x80 + position : 0xC0 + position;
107     lcd_send_byte(val, LCD_COMMAND);
108 }
109
110 static inline void lcd_char(char val) {
111     lcd_send_byte(val, LCD_CHARACTER);
112 }
113
114 void lcd_string(const char *s) {
115     while (*s) {
116         lcd_char(*s++);
117     }
118 }
119
120 void lcd_init() {
121     lcd_send_byte(0x03, LCD_COMMAND);
122     lcd_send_byte(0x03, LCD_COMMAND);

```

```

123     lcd_send_byte(0x03, LCD_COMMAND);
124     lcd_send_byte(0x02, LCD_COMMAND);
125
126     lcd_send_byte(LCD_ENTRYMODESET | LCD_ENTRYLEFT, LCD_COMMAND);
127     lcd_send_byte(LCD_FUNCTIONSET | LCD_2LINE, LCD_COMMAND);
128     lcd_send_byte(LCD_DISPLAYCONTROL | LCD_DISPLAYON, LCD_COMMAND);
129     lcd_clear();
130 }
131
132 int main() {
133     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
134         !defined(PICO_DEFAULT_I2C_SCL_PIN)
135         #warning i2c/lcd_1602_i2c example requires a board with I2C pins
136     #else
137         // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
138         i2c_init(i2c_default, 100 * 1000);
139         gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
140         gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
141         gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
142         gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
143         // Make the I2C pins available to picotool
144         bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
145             GPIO_FUNC_I2C));
146
147         lcd_init();
148
149         static char *message[] =
150             {
151                 "RP2040 by", "Raspberry Pi",
152                 "A brand new", "microcontroller",
153                 "Twin core M0", "Full C SDK",
154                 "More power in", "your product",
155                 "More beans", "than Heinz!"
156             };
157
158         while (1) {
159             for (uint m = 0; m < sizeof(message) / sizeof(message[0]); m += MAX_LINES) {
160                 for (int line = 0; line < MAX_LINES; line++) {
161                     lcd_set_cursor(line, (MAX_CHARS / 2) - strlen(message[m + line]) / 2);
162                     lcd_string(message[m + line]);
163                 }
164                 sleep_ms(2000);
165                 lcd_clear();
166             }
167         }
168     #endif
169 }

```

Bill of Materials

Table 43. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
1602A based LCD panel 3.3v	1	generic part
1602A to I2C bridge device 3.3v	1	generic part

M/M Jumper wires	4	generic part
------------------	---	--------------

Attaching a BMP280 temp/pressure sensor via I2C

This example code shows how to interface the Raspberry Pi Pico with the popular BMP280 temperature and air pressure sensor manufactured by Bosch. A similar variant, the BME280, exists that can also measure humidity. There is another example that uses the BME280 device but talks to it via SPI as opposed to I2C.

The code reads data from the sensor's registers every 500 milliseconds and prints it via the onboard UART. This example operates the BMP280 in *normal* mode, meaning that the device continuously cycles between a measurement period and a standby period at a regular interval we can set. This has the advantage that subsequent reads do not require configuration register writes and is the recommended mode of operation to filter out short-term disturbances.

 TIP

The BMP280 is highly configurable with 3 modes of operation, various oversampling levels, and 5 filter settings. Find the datasheet online (<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf>) to explore all of its capabilities beyond the simple example given here.

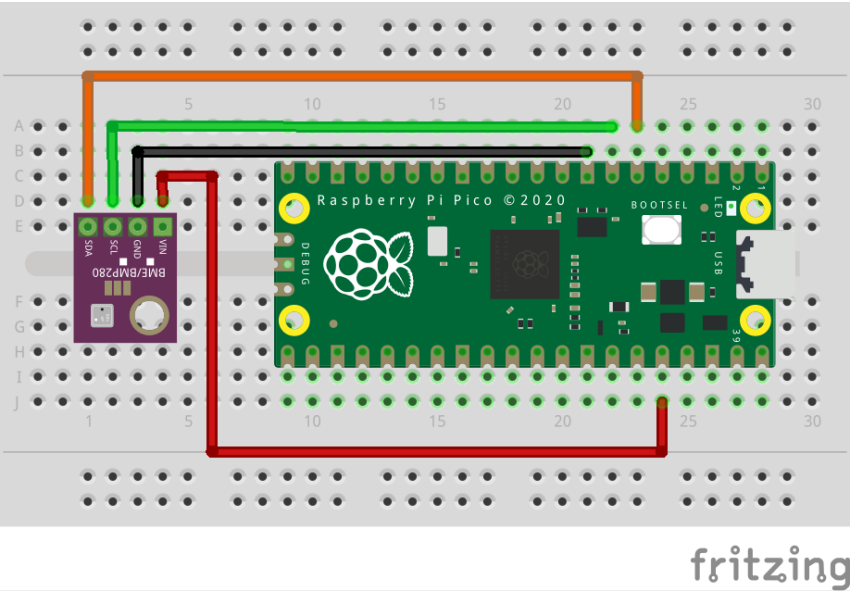
Wiring information

Wiring up the device requires 4 jumpers, to connect VCC (3.3v), GND, SDA and SCL. The example here uses the default I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3.3V pin from the Pico.

 WARNING

The BMP280 has a maximum supply voltage rating of 3.6V. Most breakout boards have voltage regulators that will allow a range of input voltages of 2-6V, but make sure to check beforehand.

Figure 19. Wiring Diagram for BMP280 sensor via I2C.



List of Files

CMakeLists.txt

CMake file to incorporate the example into the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/bmp280_i2c/CMakeLists.txt

```

1 add_executable(bmp280_i2c
2     bmp280_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(bmp280_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(bmp280_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(bmp280_i2c)

```

bmp280_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/bmp280_i2c/bmp280_i2c.c

```

1 /**
2  * Copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8
9 #include "hardware/i2c.h"
10 #include "pico/binary_info.h"
11 #include "pico/stdlib.h"
12
13 /* Example code to talk to a BMP280 temperature and pressure sensor
14
15     NOTE: Ensure the device is capable of being driven at 3.3v NOT 5v. The Pico
16     GPIO (and therefore I2C) cannot be used at 5v.
17
18     You will need to use a level shifter on the I2C lines if you want to run the
19     board at 5v.
20
21     Connections on Raspberry Pi Pico board, other boards may vary.
22
23     GPIO PICO_DEFAULT_I2C_SDA_PIN (on Pico this is GP4 (pin 6)) -> SDA on BMP280
24     board
25     GPIO PICO_DEFAULT_I2C_SCK_PIN (on Pico this is GP5 (pin 7)) -> SCL on
26     BMP280 board
27     3.3v (pin 36) -> VCC on BMP280 board
28     GND (pin 38) -> GND on BMP280 board
29 */
30
31 // device has default bus address of 0x76
32 #define ADDR _u(0x76)
33
34 // hardware registers
35 #define REG_CONFIG _u(0xF5)
36 #define REG_CTRL_MEAS _u(0xF4)
37 #define REG_RESET _u(0xE0)
38

```

```

39 #define REG_TEMP_XLSB _u(0xFC)
40 #define REG_TEMP_LSB _u(0xFB)
41 #define REG_TEMP_MSB _u(0xFA)
42
43 #define REG_PRESSURE_XLSB _u(0xF9)
44 #define REG_PRESSURE_LSB _u(0xF8)
45 #define REG_PRESSURE_MSB _u(0xF7)
46
47 // calibration registers
48 #define REG_DIG_T1_LSB _u(0x88)
49 #define REG_DIG_T1_MSB _u(0x89)
50 #define REG_DIG_T2_LSB _u(0x8A)
51 #define REG_DIG_T2_MSB _u(0x8B)
52 #define REG_DIG_T3_LSB _u(0x8C)
53 #define REG_DIG_T3_MSB _u(0x8D)
54 #define REG_DIG_P1_LSB _u(0x8E)
55 #define REG_DIG_P1_MSB _u(0x8F)
56 #define REG_DIG_P2_LSB _u(0x90)
57 #define REG_DIG_P2_MSB _u(0x91)
58 #define REG_DIG_P3_LSB _u(0x92)
59 #define REG_DIG_P3_MSB _u(0x93)
60 #define REG_DIG_P4_LSB _u(0x94)
61 #define REG_DIG_P4_MSB _u(0x95)
62 #define REG_DIG_P5_LSB _u(0x96)
63 #define REG_DIG_P5_MSB _u(0x97)
64 #define REG_DIG_P6_LSB _u(0x98)
65 #define REG_DIG_P6_MSB _u(0x99)
66 #define REG_DIG_P7_LSB _u(0x9A)
67 #define REG_DIG_P7_MSB _u(0x9B)
68 #define REG_DIG_P8_LSB _u(0x9C)
69 #define REG_DIG_P8_MSB _u(0x9D)
70 #define REG_DIG_P9_LSB _u(0x9E)
71 #define REG_DIG_P9_MSB _u(0x9F)
72
73 // number of calibration registers to be read
74 #define NUM_CALIB_PARAMS 24
75
76 struct bmp280_calib_param {
77     // temperature params
78     uint16_t dig_t1;
79     int16_t dig_t2;
80     int16_t dig_t3;
81
82     // pressure params
83     uint16_t dig_p1;
84     int16_t dig_p2;
85     int16_t dig_p3;
86     int16_t dig_p4;
87     int16_t dig_p5;
88     int16_t dig_p6;
89     int16_t dig_p7;
90     int16_t dig_p8;
91     int16_t dig_p9;
92 };
93
94 #ifdef i2c_default
95 void bmp280_init() {
96     // use the "handheld device dynamic" optimal setting (see datasheet)
97     uint8_t buf[2];
98
99     // 500ms sampling time, x16 filter
100     const uint8_t reg_config_val = ((0x04 << 5) | (0x05 << 2)) & 0xFC;
101
102     // send register number followed by its corresponding value

```

```

103     buf[0] = REG_CONFIG;
104     buf[1] = reg_config_val;
105     i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
106
107     // osrs_t x1, osrs_p x4, normal mode operation
108     const uint8_t reg_ctrl_meas_val = (0x01 << 5) | (0x03 << 2) | (0x03);
109     buf[0] = REG_CTRL_MEAS;
110     buf[1] = reg_ctrl_meas_val;
111     i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
112 }
113
114 void bmp280_read_raw(int32_t* temp, int32_t* pressure) {
115     // BMP280 data registers are auto-incrementing and we have 3 temperature and
116     // pressure registers each, so we start at 0xF7 and read 6 bytes to 0xFC
117     // note: normal mode does not require further ctrl_meas and config register writes
118
119     uint8_t buf[6];
120     uint8_t reg = REG_PRESSURE_MSB;
121     i2c_write_blocking(i2c_default, ADDR, &reg, 1, true); // true to keep master control of
bus
122     i2c_read_blocking(i2c_default, ADDR, buf, 6, false); // false - finished with bus
123
124     // store the 20 bit read in a 32 bit signed integer for conversion
125     *pressure = (buf[0] << 12) | (buf[1] << 4) | (buf[2] >> 4);
126     *temp = (buf[3] << 12) | (buf[4] << 4) | (buf[5] >> 4);
127 }
128
129 void bmp280_reset() {
130     // reset the device with the power-on-reset procedure
131     uint8_t buf[2] = { REG_RESET, 0xB6 };
132     i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
133 }
134
135 // intermediate function that calculates the fine resolution temperature
136 // used for both pressure and temperature conversions
137 int32_t bmp280_convert(int32_t temp, struct bmp280_calib_param* params) {
138     // use the 32-bit fixed point compensation implementation given in the
139     // datasheet
140
141     int32_t var1, var2;
142     var1 = (((temp >> 3) - ((int32_t)params->dig_t1 << 1))) * ((int32_t)params->dig_t2) >>
143     11;
144     var2 = (((((temp >> 4) - ((int32_t)params->dig_t1)) * ((temp >> 4) - ((int32_t)params->
145     >dig_t1))) >> 12) * ((int32_t)params->dig_t3)) >> 14;
146     return var1 + var2;
147 }
148
149 int32_t bmp280_convert_temp(int32_t temp, struct bmp280_calib_param* params) {
150     // uses the BMP280 calibration parameters to compensate the temperature value read from
its registers
151     int32_t t_fine = bmp280_convert(temp, params);
152     return (t_fine * 5 + 128) >> 8;
153 }
154
155 int32_t bmp280_convert_pressure(int32_t pressure, int32_t temp, struct bmp280_calib_param*
156     params) {
157     // uses the BMP280 calibration parameters to compensate the pressure value read from its
registers
158
159     int32_t t_fine = bmp280_convert(temp, params);
160     int32_t var1, var2;
161     uint32_t converted = 0.0;
162     var1 = (((int32_t)t_fine) >> 1) - (int32_t)64000;

```

```

161     var2 = (((var1 >> 2) * (var1 >> 2)) >> 11) * ((int32_t)params->dig_p6);
162     var2 += ((var1 * ((int32_t)params->dig_p5)) << 1);
163     var2 = (var2 >> 2) + (((int32_t)params->dig_p4) << 16);
164     var1 = (((params->dig_p3 * ((var1 >> 2) * (var1 >> 2)) >> 13)) >> 3) + (((int32_t
)params->dig_p2) * var1) >> 1)) >> 18;
165     var1 = (((32768 + var1)) * ((int32_t)params->dig_p1)) >> 15);
166     if (var1 == 0) {
167         return 0; // avoid exception caused by division by zero
168     }
169     converted = (((uint32_t)((int32_t)1048576) - pressure) - (var2 >> 12))) * 3125;
170     if (converted < 0x80000000) {
171         converted = (converted << 1) / ((uint32_t)var1);
172     } else {
173         converted = (converted / (uint32_t)var1) * 2;
174     }
175     var1 = (((int32_t)params->dig_p9) * ((int32_t)(((converted >> 3) * (converted >> 3)) >>
13))) >> 12;
176     var2 = (((int32_t)(converted >> 2)) * ((int32_t)params->dig_p8)) >> 13;
177     converted = (uint32_t)((int32_t)converted + ((var1 + var2 + params->dig_p7) >> 4));
178     return converted;
179 }
180
181 void bmp280_get_calib_params(struct bmp280_calib_param* params) {
182     // raw temp and pressure values need to be calibrated according to
183     // parameters generated during the manufacturing of the sensor
184     // there are 3 temperature params, and 9 pressure params, each with a LSB
185     // and MSB register, so we read from 24 registers
186
187     uint8_t buf[NUM_CALIB_PARAMS] = { 0 };
188     uint8_t reg = REG_DIG_T1_LSB;
189     i2c_write_blocking(i2c_default, ADDR, &reg, 1, true); // true to keep master control of
bus
190     // read in one go as register addresses auto-increment
191     i2c_read_blocking(i2c_default, ADDR, buf, NUM_CALIB_PARAMS, false); // false, we're
done reading
192
193     // store these in a struct for later use
194     params->dig_t1 = (uint16_t)(buf[1] << 8) | buf[0];
195     params->dig_t2 = (int16_t)(buf[3] << 8) | buf[2];
196     params->dig_t3 = (int16_t)(buf[5] << 8) | buf[4];
197
198     params->dig_p1 = (uint16_t)(buf[7] << 8) | buf[6];
199     params->dig_p2 = (int16_t)(buf[9] << 8) | buf[8];
200     params->dig_p3 = (int16_t)(buf[11] << 8) | buf[10];
201     params->dig_p4 = (int16_t)(buf[13] << 8) | buf[12];
202     params->dig_p5 = (int16_t)(buf[15] << 8) | buf[14];
203     params->dig_p6 = (int16_t)(buf[17] << 8) | buf[16];
204     params->dig_p7 = (int16_t)(buf[19] << 8) | buf[18];
205     params->dig_p8 = (int16_t)(buf[21] << 8) | buf[20];
206     params->dig_p9 = (int16_t)(buf[23] << 8) | buf[22];
207 }
208
209 #endif
210
211 int main() {
212     stdio_init_all();
213
214     #if !defined(I2C_DEFAULT) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
!defined(PICO_DEFAULT_I2C_SCL_PIN)
215         #warning i2c / bmp280_i2c example requires a board with I2C pins
216         puts("Default I2C pins were not defined");
217         return 0;
218     #else
219         // useful information for picotool

```

```
220     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
GPIO_FUNC_I2C));
221     bi_decl(bi_program_description("BMP280 I2C example for the Raspberry Pi Pico"));
222
223     printf("Hello, BMP280! Reading temperaure and pressure values from sensor...\n");
224
225     // I2C is "open drain", pull ups to keep signal high when no data is being sent
226     i2c_init(i2c_default, 100 * 1000);
227     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
228     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
229     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
230     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
231
232     // configure BMP280
233     bmp280_init();
234
235     // retrieve fixed compensation params
236     struct bmp280_calib_param params;
237     bmp280_get_calib_params(&params);
238
239     int32_t raw_temperature;
240     int32_t raw_pressure;
241
242     sleep_ms(250); // sleep so that data polling and register update don't collide
243     while (1) {
244         bmp280_read_raw(&raw_temperature, &raw_pressure);
245         int32_t temperature = bmp280_convert_temp(raw_temperature, &params);
246         int32_t pressure = bmp280_convert_pressure(raw_pressure, raw_temperature, &params);
247         printf("Pressure = %.3f kPa\n", pressure / 1000.f);
248         printf("Temp. = %.2f C\n", temperature / 100.f);
249         // poll every 500ms
250         sleep_ms(500);
251     }
252 #endif
253 }
```

Bill of Materials

Table 44. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
BMP280-based breakout board	1	from Pimoroni
M/M Jumper wires	4	generic part

Attaching a LIS3DH Nano Accelerometer via i2c.

This example shows you how to interface the Raspberry Pi Pico to the LIS3DH accelerometer and temperature sensor.

The code reads and displays the acceleration values of the board in the 3 axes and the ambient temperature value. The datasheet for the sensor can be found at <https://www.st.com/resource/en/datasheet/cd00274221.pdf>. The device is being operated on 'normal mode' and at a frequency of 1.344 kHz (this can be changed by editing the ODR bits of CTRL_REG4). The range of the data is controlled by the FS bit in CTRL_REG4 and is equal to ±2g in this example. The sensitivity depends on the operating mode and data range; exact values can be found on page 10 of the datasheet. In

this case, the sensitivity value is 4mg (where g is the value of gravitational acceleration on the surface of Earth). In order to use the auxiliary ADC to read temperature, we must set the BDU bit to 1 in CTRL_REG4 and the ADC_EN bit to 1 in TEMP_CFG_REG. Temperature is communicated through ADC 3.

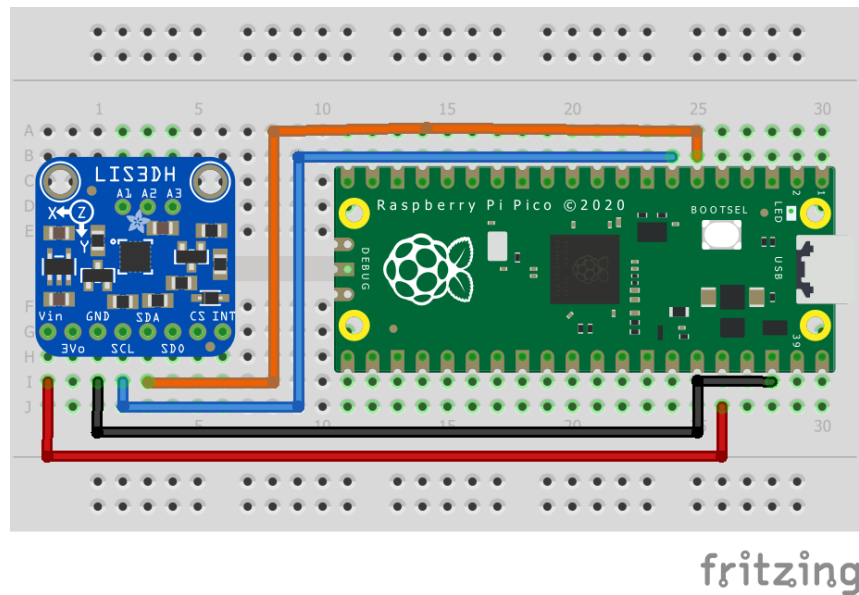
i NOTE

The sensor doesn't have features to eliminate offsets in the data and these will need to be taken into account in the code.

Wiring information

Wiring up the device requires 4 jumpers, to connect VIN, GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3V pin.

Figure 20. Wiring Diagram for LIS3DH.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/lis3dh_i2c/CMakeLists.txt

```
1 add_executable(lis3dh_i2c
2     lis3dh_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(lis3dh_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(lis3dh_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(lis3dh_i2c)
```

lis3dh_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/lis3dh_i2c/lis3dh_i2c.c

```

1  /**
2   * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3   *
4   * SPDX-License-Identifier: BSD-3-Clause
5   */
6
7  #include <stdio.h>
8  #include <string.h>
9  #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to a LIS3DH Mini GPS module.
14
15    This example reads data from all 3 axes of the accelerometer and uses an auxiliary ADC to
16    output temperature values.
17
18    Connections on Raspberry Pi Pico board, other boards may vary.
19
20    GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is 4 (physical pin 6)) -> SDA on LIS3DH board
21    GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is 5 (physical pin 7)) -> SCL on LIS3DH board
22    3.3v (physical pin 36) -> VIN on LIS3DH board
23    GND (physical pin 38) -> GND on LIS3DH board
24
25    */
26
27 // By default this device is on bus address 0x18
28
29 const int ADDRESS = 0x18;
30 const uint8_t CTRL_REG_1 = 0x20;
31 const uint8_t CTRL_REG_4 = 0x23;
32 const uint8_t TEMP_CFG_REG = 0x1F;
33
34 #ifdef i2c_default
35
36 void lis3dh_init() {
37     uint8_t buf[2];
38
39     // Turn normal mode and 1.344kHz data rate on
40     buf[0] = CTRL_REG_1;
41     buf[1] = 0x97;
42     i2c_write_blocking(i2c_default, ADDRESS, buf, 2, false);
43
44     // Turn block data update on (for temperature sensing)
45     buf[0] = CTRL_REG_4;
46     buf[1] = 0x80;
47     i2c_write_blocking(i2c_default, ADDRESS, buf, 2, false);
48
49     // Turn auxiliary ADC on
50     buf[0] = TEMP_CFG_REG;
51     buf[1] = 0xC0;
52     i2c_write_blocking(i2c_default, ADDRESS, buf, 2, false);
53 }
54
55 void lis3dh_calc_value(uint16_t raw_value, float *final_value, bool isAccel) {
56     // Convert with respect to the value being temperature or acceleration reading
57     float scaling;
58     float sensitivity = 0.004f; // g per unit
59 }

```

```

58     if (isAccel == true) {
59         scaling = 64 / sensitivity;
60     } else {
61         scaling = 64;
62     }
63
64     // raw_value is signed
65     *final_value = (float) ((int16_t) raw_value) / scaling;
66 }
67
68 void lis3dh_read_data(uint8_t reg, float *final_value, bool IsAccel) {
69     // Read two bytes of data and store in a 16 bit data structure
70     uint8_t lsb;
71     uint8_t msb;
72     uint16_t raw_accel;
73     i2c_write_blocking(i2c_default, ADDRESS, &reg, 1, true);
74     i2c_read_blocking(i2c_default, ADDRESS, &lsb, 1, false);
75
76     reg |= 0x01;
77     i2c_write_blocking(i2c_default, ADDRESS, &reg, 1, true);
78     i2c_read_blocking(i2c_default, ADDRESS, &msb, 1, false);
79
80     raw_accel = (msb << 8) | lsb;
81
82     lis3dh_calc_value(raw_accel, final_value, IsAccel);
83 }
84
85 #endif
86
87 int main() {
88     stdio_init_all();
89     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
90         !defined(PICO_DEFAULT_I2C_SCL_PIN)
91         #warning i2c/lis3dh_i2c example requires a board with I2C pins
92         puts("Default I2C pins were not defined");
93     #else
94         printf("Hello, LIS3DH! Reading raw data from registers...\n");
95
96         // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
97         i2c_init(i2c_default, 400 * 1000);
98         gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
99         gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
100        gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
101        gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
102        // Make the I2C pins available to picotool
103        bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
104        GPIO_FUNC_I2C));
105
106        float x_accel, y_accel, z_accel, temp;
107
108        lis3dh_init();
109
110        while (1) {
111            lis3dh_read_data(0x28, &x_accel, true);
112            lis3dh_read_data(0x2A, &y_accel, true);
113            lis3dh_read_data(0x2C, &z_accel, true);
114            lis3dh_read_data(0x0C, &temp, false);
115
116            // Display data
117            printf("TEMPERATURE: %.3f°C\n", temp, 176);
118            // Acceleration is read as a multiple of g (gravitational acceleration on the Earth's
119            surface)
120            printf("ACCELERATION VALUES: \n");
121            printf("X acceleration: %.3fg\n", x_accel);

```



```
119     printf("Y acceleration: %.3fg\n", y_accel);
120     printf("Z acceleration: %.3fg\n", z_accel);
121
122     sleep_ms(500);
123
124     // Clear terminal
125     printf("\033[1;1H\033[2J");
126 }
127 #endif
128 }
```

Bill of Materials

Table 45. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
LIS3DH board	1	https://www.adafruit.com/product/2809
M/M Jumper wires	4	generic part

Attaching a MCP9808 digital temperature sensor via I2C

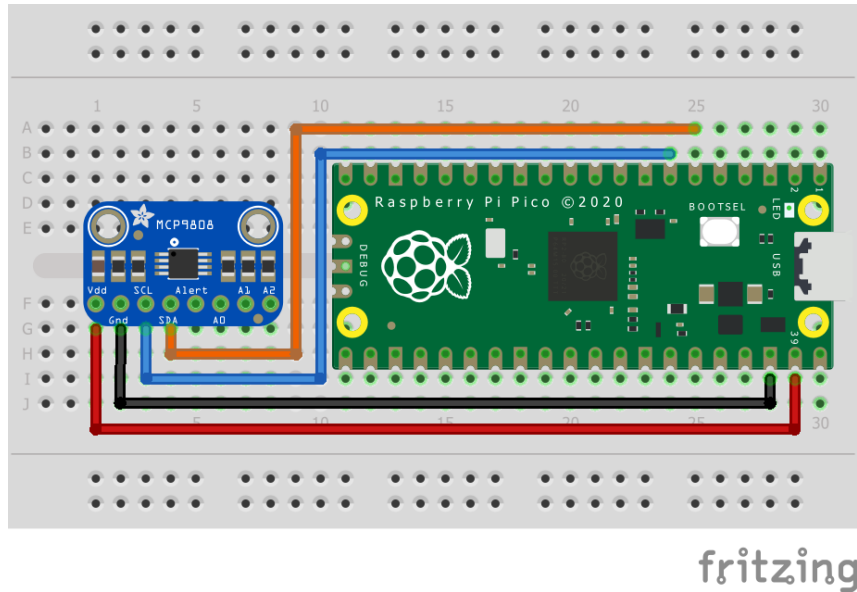
This example code shows how to interface the Raspberry Pi Pico to the MCP9808 digital temperature sensor board.

This example reads the ambient temperature value each second from the sensor and sets upper, lower and critical limits for the temperature and checks if alerts need to be raised. The CONFIG register can also be used to check for an alert if the critical temperature is surpassed.

Wiring information

Wiring up the device requires 4 jumpers, to connect VDD, GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the VSYS pin.

Figure 21. Wiring Diagram for MCP9808.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mcp9808_i2c/CMakeLists.txt

```
1 add_executable(mcp9808_i2c
2     mcp9808_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(mcp9808_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(mcp9808_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(mcp9808_i2c)
```

mcp9808_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mcp9808_i2c/mcp9808_i2c.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to a MCP9808 ±0.5°C Digital temperature Sensor
```

```

14
15  This reads and writes to registers on the board.
16
17  Connections on Raspberry Pi Pico board, other boards may vary.
18
19  GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is GP4 (physical pin 6)) -> SDA on MCP9808
   board
20  GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is GP5 (physical pin 7)) -> SCL on MCP9808
   board
21  Vsys (physical pin 39) -> VDD on MCP9808 board
22  GND (physical pin 38) -> GND on MCP9808 board
23
24  */
25  //The bus address is determined by the state of pins A0, A1 and A2 on the MCP9808 board
26  static uint8_t ADDRESS = 0x18;
27
28  //hardware registers
29
30  const uint8_t REG_POINTER = 0x00;
31  const uint8_t REG_CONFIG = 0x01;
32  const uint8_t REG_TEMP_UPPER = 0x02;
33  const uint8_t REG_TEMP_LOWER = 0x03;
34  const uint8_t REG_TEMP_CRIT = 0x04;
35  const uint8_t REG_TEMP_AMB = 0x05;
36  const uint8_t REG_RESOLUTION = 0x08;
37
38
39  void mcp9808_check_limits(uint8_t upper_byte) {
40
41      // Check flags and raise alerts accordingly
42      if ((upper_byte & 0x40) == 0x40) { //TA > TUPPER
43          printf("Temperature is above the upper temperature limit.\n");
44      }
45      if ((upper_byte & 0x20) == 0x20) { //TA < TLOWER
46          printf("Temperature is below the lower temperature limit.\n");
47      }
48      if ((upper_byte & 0x80) == 0x80) { //TA > TCRT
49          printf("Temperature is above the critical temperature limit.\n");
50      }
51  }
52
53  float mcp9808_convert_temp(uint8_t upper_byte, uint8_t lower_byte) {
54
55      float temperature;
56
57
58      //Check if TA <= 0°C and convert to denary accordingly
59      if ((upper_byte & 0x10) == 0x10) {
60          upper_byte = upper_byte & 0x0F;
61          temperature = 256 - (((float) upper_byte * 16) + ((float) lower_byte / 16));
62      } else {
63          temperature = (((float) upper_byte * 16) + ((float) lower_byte / 16));
64      }
65  }
66  return temperature;
67  }
68
69  #ifdef i2c_default
70  void mcp9808_set_limits() {
71
72      //Set an upper limit of 30°C for the temperature
73      uint8_t upper_temp_msb = 0x01;
74      uint8_t upper_temp_lsb = 0xE0;
75

```

```

76 //Set a lower limit of 20°C for the temperature
77 uint8_t lower_temp_msb = 0x01;
78 uint8_t lower_temp_lsb = 0x40;
79
80 //Set a critical limit of 40°C for the temperature
81 uint8_t crit_temp_msb = 0x02;
82 uint8_t crit_temp_lsb = 0x80;
83
84 uint8_t buf[3];
85 buf[0] = REG_TEMP_UPPER;
86 buf[1] = upper_temp_msb;
87 buf[2] = upper_temp_lsb;
88 i2c_write_blocking(i2c_default, ADDRESS, buf, 3, false);
89
90 buf[0] = REG_TEMP_LOWER;
91 buf[1] = lower_temp_msb;
92 buf[2] = lower_temp_lsb;
93 i2c_write_blocking(i2c_default, ADDRESS, buf, 3, false);
94
95 buf[0] = REG_TEMP_CRIT;
96 buf[1] = crit_temp_msb;
97 buf[2] = crit_temp_lsb;
98 i2c_write_blocking(i2c_default, ADDRESS, buf, 3, false);
99 }
100 #endif
101
102 int main() {
103
104     stdio_init_all();
105
106     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
107         !defined(PICO_DEFAULT_I2C_SCL_PIN)
108     #warning i2c/mcp9808_i2c example requires a board with I2C pins
109     puts("Default I2C pins were not defined");
110 #else
111     printf("Hello, MCP9808! Reading raw data from registers...\n");
112
113     // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
114     i2c_init(i2c_default, 400 * 1000);
115     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
116     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
117     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
118     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
119     // Make the I2C pins available to picotool
120     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
121                               GPIO_FUNC_I2C));
122
123     mcp9808_set_limits();
124
125     uint8_t buf[2];
126     uint16_t upper_byte;
127     uint16_t lower_byte;
128
129     float temperature;
130
131     while (1) {
132         // Start reading ambient temperature register for 2 bytes
133         i2c_write_blocking(i2c_default, ADDRESS, &REG_TEMP_AMB, 1, true);
134         i2c_read_blocking(i2c_default, ADDRESS, buf, 2, false);
135
136         upper_byte = buf[0];
137         lower_byte = buf[1];
138
139         //isolates limit flags in upper byte

```

```
138     mcp9808_check_limits(upper_byte & 0xE0);
139
140     //clears flag bits in upper byte
141     temperature = mcp9808_convert_temp(upper_byte & 0x1F, lower_byte);
142     printf("Ambient temperature: %.4f°C\n", temperature);
143
144     sleep_ms(1000);
145 }
146 #endif
147 }
```

Bill of Materials

Table 46. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
MCP9808 board	1	https://www.adafruit.com/product/1782
M/M Jumper wires	4	generic part

Attaching a MMA8451 3-axis digital accelerometer via I2C

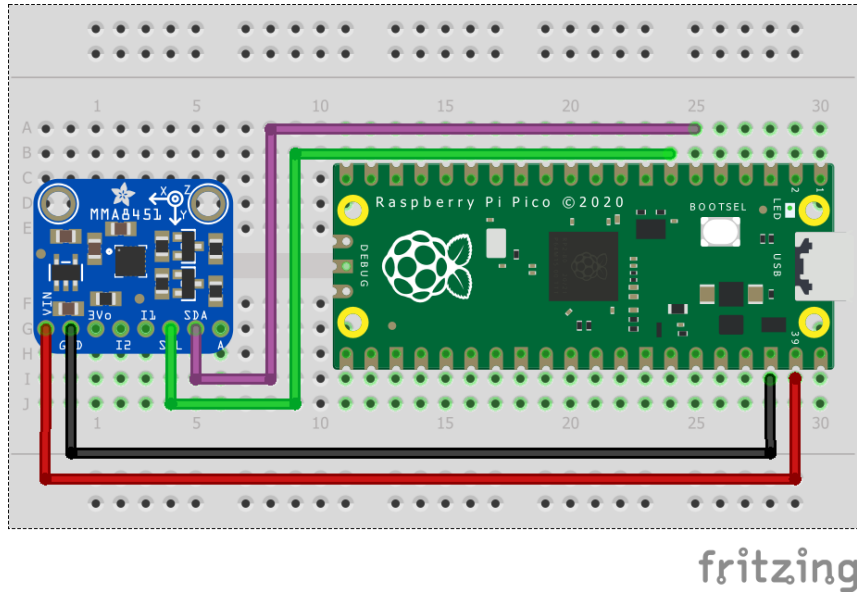
This example code shows how to interface the Raspberry Pi Pico to the MMA8451 digital accelerometer sensor board.

This example reads and displays the acceleration values of the board in the 3 axis. It also allows the user to set the trade-off between the range and precision based on the values they require. Values often have an offset which can be accounted for by writing to the offset correction registers. The datasheet for the sensor can be found at <https://cdn-shop.adafruit.com/datasheets/MMA8451Q-1.pdf> for additional information.

Wiring information

Wiring up the device requires 4 jumpers, to connect VIN, GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the VSYS pin.

Figure 22. Wiring
Diagram for
MMA8451.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mma8451_i2c/CMakeLists.txt

```
1 add_executable(mma8451_i2c
2     mma8451_i2c.c
3 )
4 # pull in common dependencies and additional i2c hardware support
5 target_link_libraries(mma8451_i2c pico_stdlib hardware_i2c)
6
7 # create map/bin/hex file etc.
8 pico_add_extra_outputs(mma8451_i2c)
9
10 # add url via pico_set_program_url
11 example_auto_set_url(mma8451_i2c)
```

mma8451_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mma8451_i2c/mma8451_i2c.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to a MMA8451 triple-axis accelerometer.
14  */
```

```

15  This reads and writes to registers on the board.
16
17  Connections on Raspberry Pi Pico board, other boards may vary.
18
19  GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is GP4 (physical pin 6)) -> SDA on MMA8451
   board
20  GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is GP5 (physical pin 7)) -> SCL on MMA8451
   board
21  VSYS (physical pin 39) -> VDD on MMA8451 board
22  GND (physical pin 38) -> GND on MMA8451 board
23
24  */
25
26  const uint8_t ADDRESS = 0x1D;
27
28  //hardware registers
29
30  const uint8_t REG_X_MSB = 0x01;
31  const uint8_t REG_X_LSB = 0x02;
32  const uint8_t REG_Y_MSB = 0x03;
33  const uint8_t REG_Y_LSB = 0x04;
34  const uint8_t REG_Z_MSB = 0x05;
35  const uint8_t REG_Z_LSB = 0x06;
36  const uint8_t REG_DATA_CFG = 0x0E;
37  const uint8_t REG_CTRL_REG1 = 0x2A;
38
39  // Set the range and precision for the data
40  const uint8_t range_config = 0x01; // 0x00 for ±2g, 0x01 for ±4g, 0x02 for ±8g
41  const float count = 2048; // 4096 for ±2g, 2048 for ±4g, 1024 for ±8g
42
43  uint8_t buf[2];
44
45  float mma8451_convert_accel(uint16_t raw_accel) {
46      float acceleration;
47      // Acceleration is read as a multiple of g (gravitational acceleration on the Earth's
   surface)
48      // Check if acceleration < 0 and convert to decimal accordingly
49      if ((raw_accel & 0x2000) == 0x2000) {
50          raw_accel &= 0x1FFF;
51          acceleration = (-8192 + (float) raw_accel) / count;
52      } else {
53          acceleration = (float) raw_accel / count;
54      }
55      acceleration *= 9.81f;
56      return acceleration;
57  }
58
59  #ifdef i2c_default
60  void mma8451_set_state(uint8_t state) {
61      buf[0] = REG_CTRL_REG1;
62      buf[1] = state; // Set RST bit to 1
63      i2c_write_blocking(i2c_default, ADDRESS, buf, 2, false);
64  }
65  #endif
66
67  int main() {
68      stdio_init_all();
69
70  #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
   !defined(PICO_DEFAULT_I2C_SCK_PIN)
71  #warning i2c/mma8451_i2c example requires a board with I2C pins
72      puts("Default I2C pins were not defined");
73  #else
74      printf("Hello, MMA8451! Reading raw data from registers...\n");

```

```

75
76 // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
77 i2c_init(i2c_default, 400 * 1000);
78 gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
79 gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
80 gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
81 gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
82 // Make the I2C pins available to picotool
83 bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
GPIO_FUNC_I2C));
84
85 float x_acceleration;
86 float y_acceleration;
87 float z_acceleration;
88
89 // Enable standby mode
90 mma8451_set_state(0x00);
91
92 // Edit configuration while in standby mode
93 buf[0] = REG_DATA_CFG;
94 buf[1] = range_config;
95 i2c_write_blocking(i2c_default, ADDRESS, buf, 2, false);
96
97 // Enable active mode
98 mma8451_set_state(0x01);
99
100 while (1) {
101
102 // Start reading acceleration registers for 2 bytes
103 i2c_write_blocking(i2c_default, ADDRESS, &REG_X_MSB, 1, true);
104 i2c_read_blocking(i2c_default, ADDRESS, buf, 2, false);
105 x_acceleration = mma8451_convert_accel(buf[0] << 6 | buf[1] >> 2);
106
107 i2c_write_blocking(i2c_default, ADDRESS, &REG_Y_MSB, 1, true);
108 i2c_read_blocking(i2c_default, ADDRESS, buf, 2, false);
109 y_acceleration = mma8451_convert_accel(buf[0] << 6 | buf[1] >> 2);
110
111 i2c_write_blocking(i2c_default, ADDRESS, &REG_Z_MSB, 1, true);
112 i2c_read_blocking(i2c_default, ADDRESS, buf, 2, false);
113 z_acceleration = mma8451_convert_accel(buf[0] << 6 | buf[1] >> 2);
114
115 // Display acceleration values
116 printf("ACCELERATION VALUES: \n");
117 printf("X acceleration: %.6fms^-2\n", x_acceleration);
118 printf("Y acceleration: %.6fms^-2\n", y_acceleration);
119 printf("Z acceleration: %.6fms^-2\n", z_acceleration);
120
121 sleep_ms(500);
122
123 // Clear terminal
124 printf("\033[1;1H\033[2J");
125 }
126
127 #endif
128 }

```

Bill of Materials

Table 47. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
MMA8451 board	1	https://www.adafruit.com/product/2019
M/M Jumper wires	4	generic part

Attaching an MPL3115A2 altimeter via I2C

This example code shows how to interface the Raspberry Pi Pico to an MPL3115A2 altimeter via I2C. The MPL3115A2 has onboard pressure and temperature sensors which are used to estimate the altitude. In comparison to the BMP-family of pressure and temperature sensors, the MPL3115A2 has two interrupt pins for ultra low power operation and takes care of the sensor reading compensation on the board! It also has multiple modes of operation and impressive operating conditions.

The board used in this example [comes from Adafruit](#), but any MPL3115A2 breakouts should work similarly.

The MPL3115A2 makes available two ways of reading its temperature and pressure data. The first is known as polling, where the Pico will continuously read data out of a set of auto-incrementing registers which are refreshed with new data every so often. The second, which this example will demonstrate, uses a 160-byte first-in-first-out (FIFO) queue and configurable interrupts to tell the Pico when to read data. More information regarding when the interrupts can be triggered available [in the datasheet](#). This example waits for the 32 sample FIFO to overflow, detects this via an interrupt pin, and then averages the 32 samples taken. The sensor is configured to take a sample every second.

Bit math is used to convert the temperature and altitude data from the raw bits collected in the registers. Take the temperature calculation as an example: it is a 12-bit signed number with 8 integer bits and 4 fractional bits. First, we read the 2 8-bit registers and store them in a buffer. Then, we concatenate them into one unsigned 16-bit integer starting with the OUT_T_MSB register, thus making sure that the last bit of this register is aligned with the MSB in our 16 bit unsigned integer so it is correctly interpreted as the signed bit when we later cast this to a signed 16-bit integer. Finally, the entire number is converted to a float implicitly when we multiply it by $1/2^8$ to shift it 8 bits to the right of the decimal point. Though only the last 4 bits of the OUT_T_LSB register hold data, this does not matter as the remaining 4 are held at zero and "disappear" when we shift the decimal point left by 8. Similar logic is applied to the altitude calculation.

TIP

Choosing the right sensor for your project among so many choices can be hard! There are multiple factors you may have to consider in addition to any constraints imposed on you. Cost, operating temperature, sensor resolution, power consumption, ease of use, communication protocols and supply voltage are all but a few factors that can play a role in sensor choice. For most hobbyist purposes though, the majority of sensors out there will do just fine!

Wiring information

Wiring up the device requires 5 jumpers, to connect VCC (3.3v), GND, INT1, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and GPIO 5 (SCL) by default. Power is supplied from the 3.3V pin.

The MPL3115A2 has a 1.6-3.6V voltage supply range. This means it can work with the Pico's 3.3v pins out of the box but our Adafruit breakout has an onboard voltage regulator for good measure. This may not always be true of other sensors, though.

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

```
1 add_executable(mpl3115a2_i2c
2     mpl3115a2_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(mpl3115a2_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(mpl3115a2_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(mpl3115a2_i2c)
```

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/mp13115a2_i2c/mp13115a2_i2c.c

```
1 /**
2  * Copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
```

```

6
7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "pico/binary_info.h"
10 #include "hardware/gpio.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to an MPL3115A2 altimeter sensor via I2C
14
15     See accompanying documentation in README.adoc or the C++ SDK booklet.
16
17     Connections on Raspberry Pi Pico board, other boards may vary.
18
19     GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is 4 (pin 6)) -> SDA on MPL3115A2 board
20     GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is 5 (pin 7)) -> SCL on MPL3115A2 board
21     GPIO 16 -> INT1 on MPL3115A2 board
22     3.3v (pin 36) -> VCC on MPL3115A2 board
23     GND (pin 38) -> GND on MPL3115A2 board
24 */
25
26 // 7-bit address
27 #define ADDR 0x60
28 #define INT1_PIN _u(16)
29
30 // following definitions only valid for F_MODE > 0 (ie. if FIFO enabled)
31 #define MPL3115A2_F_DATA _u(0x01)
32 #define MPL3115A2_F_STATUS _u(0x00)
33 #define MPL3115A2_F_SETUP _u(0x0F)
34 #define MPL3115A2_INT_SOURCE _u(0x12)
35 #define MPL3115A2_CTRLREG1 _u(0x26)
36 #define MPL3115A2_CTRLREG2 _u(0x27)
37 #define MPL3115A2_CTRLREG3 _u(0x28)
38 #define MPL3115A2_CTRLREG4 _u(0x29)
39 #define MPL3115A2_CTRLREG5 _u(0x2A)
40 #define MPL3115A2_PT_DATA_CFG _u(0x13)
41 #define MPL3115A2_OFF_P _u(0x2B)
42 #define MPL3115A2_OFF_T _u(0x2C)
43 #define MPL3115A2_OFF_H _u(0x2D)
44
45 #define MPL3115A2_FIFO_DISABLED _u(0x00)
46 #define MPL3115A2_FIFO_STOP_ON_OVERFLOW _u(0x80)
47 #define MPL3115A2_FIFO_SIZE 32
48 #define MPL3115A2_DATA_BATCH_SIZE 5
49 #define MPL3115A2_ALTITUDE_NUM_REGS 3
50 #define MPL3115A2_ALTITUDE_INT_SIZE 20
51 #define MPL3115A2_TEMPERATURE_INT_SIZE 12
52 #define MPL3115A2_NUM_FRAC_BITS 4
53
54 #define PARAM_ASSERTIONS_ENABLE_I2C 1
55
56 volatile uint8_t fifo_data[MPL3115A2_FIFO_SIZE * MPL3115A2_DATA_BATCH_SIZE];
57 volatile bool has_new_data = false;
58
59 struct mpl3115a2_data_t {
60     // Q8.4 fixed point
61     float temperature;
62     // Q16.4 fixed-point
63     float altitude;
64 };
65
66 void copy_to_vbuf(uint8_t buf1[], volatile uint8_t buf2[], uint buflen) {
67     for (size_t i = 0; i < buflen; i++) {
68         buf2[i] = buf1[i];
69     }

```

```

70 }
71
72 #ifdef i2c_default
73
74 void mpl3115a2_read_fifo(volatile uint8_t fifo_buf[]) {
75     // drains the 160 byte FIFO
76     uint8_t reg = MPL3115A2_F_DATA;
77     uint8_t buf[MPL3115A2_FIFO_SIZE * MPL3115A2_DATA_BATCH_SIZE];
78     i2c_write_blocking(i2c_default, ADDR, &reg, 1, true);
79     // burst read 160 bytes from fifo
80     i2c_read_blocking(i2c_default, ADDR, buf, MPL3115A2_FIFO_SIZE *
        MPL3115A2_DATA_BATCH_SIZE, false);
81     copy_to_vbuf(buf, fifo_buf, MPL3115A2_FIFO_SIZE * MPL3115A2_DATA_BATCH_SIZE);
82 }
83
84 uint8_t mpl3115a2_read_reg(uint8_t reg) {
85     uint8_t read;
86     i2c_write_blocking(i2c_default, ADDR, &reg, 1, true); // keep control of bus
87     i2c_read_blocking(i2c_default, ADDR, &read, 1, false);
88     return read;
89 }
90
91 void mpl3115a2_init() {
92     // set as altimeter with oversampling ratio of 128
93     uint8_t buf[] = {MPL3115A2_CTRLREG1, 0xB8};
94     i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
95
96     // set data refresh every 2 seconds, 0 next bits as we're not using those interrupts
97     buf[0] = MPL3115A2_CTRLREG2, buf[1] = 0x00;
98     i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
99
100    // set both interrupts pins to active low and enable internal pullups
101    buf[0] = MPL3115A2_CTRLREG3, buf[1] = 0x01;
102    i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
103
104    // enable FIFO interrupt
105    buf[0] = MPL3115A2_CTRLREG4, buf[1] = 0x40;
106    i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
107
108    // tie FIFO interrupt to pin INT1
109    buf[0] = MPL3115A2_CTRLREG5, buf[1] = 0x40;
110    i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
111
112    // set p, t and h offsets here if needed
113    // eg. 2's complement number: 0xFF subtracts 1 meter
114    //buf[0] = MPL3115A2_OFF_H, buf[1] = 0xFF;
115    //i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
116
117    // do not accept more data on FIFO overflow
118    buf[0] = MPL3115A2_F_SETUP, buf[1] = MPL3115A2_FIFO_STOP_ON_OVERFLOW;
119    i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
120
121    // set device active
122    buf[0] = MPL3115A2_CTRLREG1, buf[1] = 0xB9;
123    i2c_write_blocking(i2c_default, ADDR, buf, 2, false);
124 }
125
126 void gpio_callback(uint gpio, __unused uint32_t events) {
127     // if we had enabled more than 2 interrupts on same pin, then we should read
128     // INT_SOURCE reg to find out which interrupt triggered
129
130     // we can filter by which GPIO was triggered
131     if (gpio == INT1_PIN) {
132         // FIFO overflow interrupt

```

```

133     // watermark bits set to 0 in F_SETUP reg, so only possible event is an overflow
134     // otherwise, we would read F_STATUS to confirm it was an overflow
135     printf("FIFO overflow!\n");
136     // drain the fifo
137     mpl3115a2_read_fifo(fifo_data);
138     // read status register to clear interrupt bit
139     mpl3115a2_read_reg(MPL3115A2_F_STATUS);
140     has_new_data = true;
141 }
142 }
143
144 #endif
145
146 void mpl3115a2_convert_fifo_batch(uint8_t start, volatile uint8_t buf[], struct
    mpl3115a2_data_t *data) {
147     // convert a batch of fifo data into temperature and altitude data
148
149     // 3 altitude registers: MSB (8 bits), CSB (8 bits) and LSB (4 bits, starting from MSB)
150     // first two are integer bits (2's complement) and LSB is fractional bits -> makes 20 bit
    signed integer
151     int32_t h = (int32_t) buf[start] << 24;
152     h |= (int32_t) buf[start + 1] << 16;
153     h |= (int32_t) buf[start + 2] << 8;
154     data->altitude = ((float)h) / 65536.f;
155
156     // 2 temperature registers: MSB (8 bits) and LSB (4 bits, starting from MSB)
157     // first 8 are integer bits with sign and LSB is fractional bits -> 12 bit signed integer
158     int16_t t = (int16_t) buf[start + 3] << 8;
159     t |= (int16_t) buf[start + 4];
160     data->temperature = ((float)t) / 256.f;
161 }
162
163 int main() {
164     stdio_init_all();
165     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
        !defined(PICO_DEFAULT_I2C_SCL_PIN)
166     #warning i2c / mpl3115a2_i2c example requires a board with I2C pins
167     puts("Default I2C pins were not defined");
168     return 0;
169     #else
170     printf("Hello, MPL3115A2. Waiting for something to interrupt me!...\n");
171
172     // use default I2C0 at 400kHz, I2C is active low
173     i2c_init(i2c_default, 400 * 1000);
174     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
175     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
176     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
177     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
178
179     gpio_init(INT1_PIN);
180     gpio_pull_up(INT1_PIN); // pull it up even more!
181
182     // add program information for picotool
183     bi_decl(bi_program_name("Example in the pico-examples library for the MPL3115A2
        altimeter"));
184     bi_decl(bi_1pin_with_name(16, "Interrupt pin 1"));
185     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
        GPIO_FUNC_I2C));
186
187     mpl3115a2_init();
188
189     gpio_set_irq_enabled_with_callback(INT1_PIN, GPIO_IRQ_LEVEL_LOW, true, &gpio_callback);
190
191     while (1) {

```

```
192 // as interrupt data comes in, let's print the 32 sample average
193 if (has_new_data) {
194     float tsum = 0, hsum = 0;
195     struct mpl3115a2_data_t data;
196     for (int i = 0; i < MPL3115A2_FIFO_SIZE; i++) {
197         mpl3115a2_convert_fifo_batch(i * MPL3115A2_DATA_BATCH_SIZE, fifo_data, &
198         data);
199         tsum += data.temperature;
200         hsum += data.altitude;
201     }
202     printf("%d sample average -> t: %.4f C, h: %.4f m\n", MPL3115A2_FIFO_SIZE, tsum
203     / MPL3115A2_FIFO_SIZE, hsum / MPL3115A2_FIFO_SIZE);
204     has_new_data = false;
205 }
206 sleep_ms(10);
207 };
208 #endif
209 }
```

Bill of Materials

Table 48. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
MPL3115A2 altimeter	1	Adafruit
M/M Jumper wires	5	generic part

Attaching an OLED display via I2C

This example code shows how to interface the Raspberry Pi Pico with an 128x32 OLED display board based on the SSD1306 display driver, datasheet [here](#).

The code displays a series of small demo graphics; tiny raspberries that scroll horizontally, some text, and some line drawing, in the process showing you how to initialize the display, write to the entire display, write to only a portion of the display, configure scrolling, invert the display etc.

The SSD1306 is operated via a list of versatile commands (see datasheet) that allows the user to access all the capabilities of the driver. After sending a slave address, the data that follows can be either a command, flags to follow up a command or data to be written directly into the display’s RAM. A control byte is required for each write after the slave address so that the driver knows what type of data is being sent.

The example code supports displays of 32 pixel or 64 pixels high by 128 pixels wide by changing a define at the top of the code.

In the 32 vertical pixels case, the display is partitioned into 4 pages, each 8 pixels in height. In RAM, this looks roughly like:

	COL0	COL1	COL2	COL3	...	COL126	COL127
PAGE 0							
PAGE 1							

PAGE 2									
PAGE 3									

Within each page, we have:

	COL0	COL1	COL2	COL3	...	COL126	COL127
COM 0							
COM 1							
:							
COM 7							

NOTE

There is a difference between columns in RAM and the actual segment pads that connect the driver to the display. The RAM addresses COL0 - COL127 are mapped to these segment pins SEG0 - SEG127 by default. The distinction between these two is important as we can for example, easily mirror contents of RAM without rewriting a buffer.

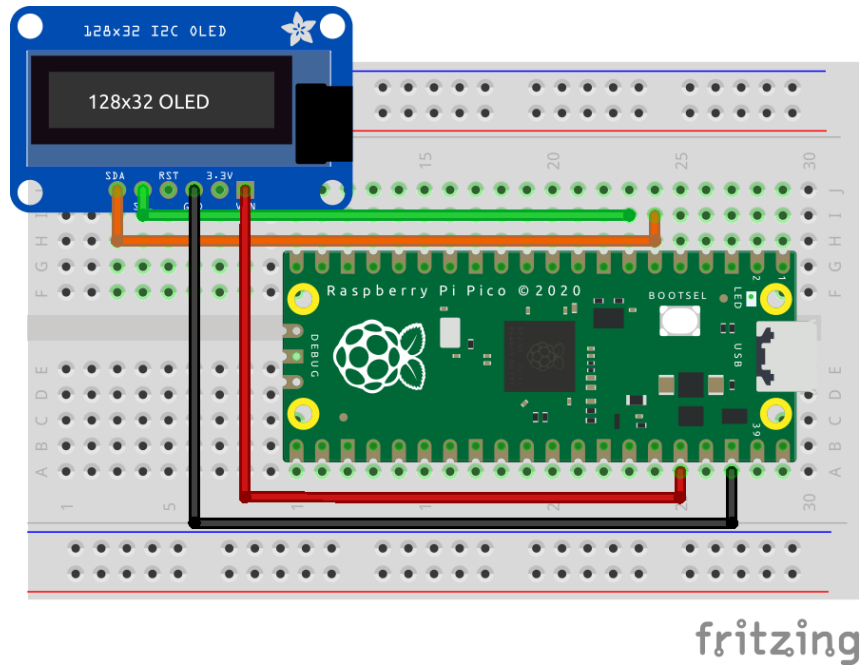
The driver has 3 modes of transferring the pixels in RAM to the display (provided that the driver is set to use its RAM content to drive the display, ie. command 0xA4 is sent). We choose horizontal addressing mode which, after setting the column address and page address registers to our desired start positions, will increment the column address register until the OLED display width is reached (127 in our case) after which the column address register will reset to its starting value and the page address is incremented. Once the page register reaches the end, it will wrap around as well. Effectively, this scans across the display from top to bottom, left to right in blocks that are 8 pixels high. When a byte is sent to be written into RAM, it sets all the rows for the current position of the column address register. So, if we send 10101010, and we are on PAGE 0 and COL1, COM0 is set to 1, COM1 is set to 0, COM2 is set to 1, and so on. Effectively, the byte is "transposed" to fill a single page's column. The datasheet has further information on this and the two other modes.

Horizontal addressing mode has the key advantage that we can keep one single 512 byte buffer (128 columns x 4 pages and each byte fills a page's rows) and write this in one go to the RAM (column address auto increments on writes as well as reads) instead of working with 2D matrices of pixels and adding more overhead.

Wiring information

Wiring up the device requires 4 jumpers, to connect VCC (3.3v), GND, SDA and SCL and optionally a 5th jumper for the driver RESET pin. The example here uses the default I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3.3V pin from the Pico.

Figure 24. Wiring Diagram for oled display via I2C.



List of Files

CMakeLists.txt

CMake file to incorporate the example into the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/ssd1306_i2c/CMakeLists.txt

```
1 add_executable(ssd1306_i2c
2     ssd1306_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(ssd1306_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(ssd1306_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(ssd1306_i2c)
```

ssd1306_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/ssd1306_i2c/ssd1306_i2c.c

```
1 /**
2  * Copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include <stdlib.h>
10 #include <ctype.h>
```



```

11 #include "pico/stdlib.h"
12 #include "pico/binary_info.h"
13 #include "hardware/i2c.h"
14 #include "raspberrypi26x32.h"
15 #include "ssd1306_font.h"
16
17 /* Example code to talk to an SSD1306-based OLED display
18
19     The SSD1306 is an OLED/PLED driver chip, capable of driving displays up to
20     128x64 pixels.
21
22     NOTE: Ensure the device is capable of being driven at 3.3v NOT 5v. The Pico
23     GPIO (and therefore I2C) cannot be used at 5v.
24
25     You will need to use a level shifter on the I2C lines if you want to run the
26     board at 5v.
27
28     Connections on Raspberry Pi Pico board, other boards may vary.
29
30     GPIO PICO_DEFAULT_I2C_SDA_PIN (on Pico this is GP4 (pin 6)) -> SDA on display
31     board
32     GPIO PICO_DEFAULT_I2C_SCL_PIN (on Pico this is GP5 (pin 7)) -> SCL on
33     display board
34     3.3v (pin 36) -> VCC on display board
35     GND (pin 38) -> GND on display board
36 */
37
38 // Define the size of the display we have attached. This can vary, make sure you
39 // have the right size defined or the output will look rather odd!
40 // Code has been tested on 128x32 and 128x64 OLED displays
41 #define SSD1306_HEIGHT      32
42 #define SSD1306_WIDTH       128
43
44 #define SSD1306_I2C_ADDR     _u(0x3C)
45
46 // 400 is usual, but often these can be overclocked to improve display response.
47 // Tested at 1000 on both 32 and 84 pixel height devices and it worked.
48 #define SSD1306_I2C_CLK      400
49 // #define SSD1306_I2C_CLK      1000
50
51
52 // commands (see datasheet)
53 #define SSD1306_SET_MEM_MODE    _u(0x20)
54 #define SSD1306_SET_COL_ADDR    _u(0x21)
55 #define SSD1306_SET_PAGE_ADDR    _u(0x22)
56 #define SSD1306_SET_HORIZ_SCROLL _u(0x26)
57 #define SSD1306_SET_SCROLL      _u(0x2E)
58
59 #define SSD1306_SET_DISP_START_LINE _u(0x40)
60
61 #define SSD1306_SET_CONTRAST     _u(0x81)
62 #define SSD1306_SET_CHARGE_PUMP _u(0x8D)
63
64 #define SSD1306_SET_SEG_REMAP    _u(0xA0)
65 #define SSD1306_SET_ENTIRE_ON    _u(0xA4)
66 #define SSD1306_SET_ALL_ON      _u(0xA5)
67 #define SSD1306_SET_NORM_DISP    _u(0xA6)
68 #define SSD1306_SET_INV_DISP     _u(0xA7)
69 #define SSD1306_SET_MUX_RATIO    _u(0xA8)
70 #define SSD1306_SET_DISP         _u(0xAE)
71 #define SSD1306_SET_COM_OUT_DIR  _u(0xC0)
72 #define SSD1306_SET_COM_OUT_DIR_FLIP _u(0xC0)
73
74 #define SSD1306_SET_DISP_OFFSET  _u(0xD3)

```

```

75 #define SSD1306_SET_DISP_CLK_DIV    _u(0xD5)
76 #define SSD1306_SET_PRECHARGE       _u(0xD9)
77 #define SSD1306_SET_COM_PIN_CFG      _u(0xDA)
78 #define SSD1306_SET_VCOM_DESEL       _u(0xDB)
79
80 #define SSD1306_PAGE_HEIGHT          _u(8)
81 #define SSD1306_NUM_PAGES             (SSD1306_HEIGHT / SSD1306_PAGE_HEIGHT)
82 #define SSD1306_BUF_LEN              (SSD1306_NUM_PAGES * SSD1306_WIDTH)
83
84 #define SSD1306_WRITE_MODE            _u(0xFE)
85 #define SSD1306_READ_MODE             _u(0xFF)
86
87
88 struct render_area {
89     uint8_t start_col;
90     uint8_t end_col;
91     uint8_t start_page;
92     uint8_t end_page;
93
94     int buflen;
95 };
96
97 void calc_render_area_buflen(struct render_area *area) {
98     // calculate how long the flattened buffer will be for a render area
99     area->buflen = (area->end_col - area->start_col + 1) * (area->end_page - area->start_page + 1);
100 }
101
102 #ifdef i2c_default
103
104 void SSD1306_send_cmd(uint8_t cmd) {
105     // I2C write process expects a control byte followed by data
106     // this "data" can be a command or data to follow up a command
107     // Co = 1, D/C = 0 => the driver expects a command
108     uint8_t buf[2] = {0x80, cmd};
109     i2c_write_blocking(i2c_default, SSD1306_I2C_ADDR, buf, 2, false);
110 }
111
112 void SSD1306_send_cmd_list(uint8_t *buf, int num) {
113     for (int i=0; i<num; i++)
114         SSD1306_send_cmd(buf[i]);
115 }
116
117 void SSD1306_send_buf(uint8_t buf[], int buflen) {
118     // in horizontal addressing mode, the column address pointer auto-increments
119     // and then wraps around to the next page, so we can send the entire frame
120     // buffer in one goooooo!
121
122     // copy our frame buffer into a new buffer because we need to add the control byte
123     // to the beginning
124
125     uint8_t *temp_buf = malloc(buflen + 1);
126
127     temp_buf[0] = 0x40;
128     memcpy(temp_buf+1, buf, buflen);
129
130     i2c_write_blocking(i2c_default, SSD1306_I2C_ADDR, temp_buf, buflen + 1, false);
131
132     free(temp_buf);
133 }
134
135 void SSD1306_init() {
136     // Some of these commands are not strictly necessary as the reset
137     // process defaults to some of these but they are shown here

```

```

138 // to demonstrate what the initialization sequence looks like
139 // Some configuration values are recommended by the board manufacturer
140
141 uint8_t cmds[] = {
142     SSD1306_SET_DISP,           // set display off
143     /* memory mapping */
144     SSD1306_SET_MEM_MODE,       // set memory address mode 0 = horizontal, 1 =
vertical, 2 = page
145     0x00,                       // horizontal addressing mode
146     /* resolution and layout */
147     SSD1306_SET_DISP_START_LINE, // set display start line to 0
148     SSD1306_SET_SEG_REMAP | 0x01, // set segment re-map, column address 127 is mapped
to SEG0
149     SSD1306_SET_MUX_RATIO,       // set multiplex ratio
150     SSD1306_HEIGHT - 1,         // Display height - 1
151     SSD1306_SET_COM_OUT_DIR | 0x08, // set COM (common) output scan direction. Scan from
bottom up, COM[N-1] to COM0
152     SSD1306_SET_DISP_OFFSET,     // set display offset
153     0x00,                       // no offset
154     SSD1306_SET_COM_PIN_CFG,     // set COM (common) pins hardware configuration.
Board specific magic number.
155                                // 0x02 Works for 128x32, 0x12 Possibly works for
128x64. Other options 0x22, 0x32
156 #if ((SSD1306_WIDTH == 128) && (SSD1306_HEIGHT == 32))
157     0x02,
158 #elif ((SSD1306_WIDTH == 128) && (SSD1306_HEIGHT == 64))
159     0x12,
160 #else
161     0x02,
162 #endif
163     /* timing and driving scheme */
164     SSD1306_SET_DISP_CLK_DIV,    // set display clock divide ratio
165     0x80,                       // div ratio of 1, standard freq
166     SSD1306_SET_PRECHARGE,       // set pre-charge period
167     0xF1,                       // Vcc internally generated on our board
168     SSD1306_SET_VCOM_DESEL,      // set VCOMH deselect level
169     0x30,                       // 0.83xVcc
170     /* display */
171     SSD1306_SET_CONTRAST,        // set contrast control
172     0xFF,
173     SSD1306_SET_ENTIRE_ON,        // set entire display on to follow RAM content
174     SSD1306_SET_NORM_DISP,       // set normal (not inverted) display
175     SSD1306_SET_CHARGE_PUMP,     // set charge pump
176     0x14,                       // Vcc internally generated on our board
177     SSD1306_SET_SCROLL | 0x00,   // deactivate horizontal scrolling if set. This is
necessary as memory writes will corrupt if scrolling was enabled
178     SSD1306_SET_DISP | 0x01, // turn display on
179 };
180
181 SSD1306_send_cmd_list(cmds, count_of(cmds));
182 }
183
184 void SSD1306_scroll(bool on) {
185     // configure horizontal scrolling
186     uint8_t cmds[] = {
187         SSD1306_SET_HORIZ_SCROLL | 0x00,
188         0x00, // dummy byte
189         0x00, // start page 0
190         0x00, // time interval
191         0x03, // end page 3 SSD1306_NUM_PAGES ??
192         0x00, // dummy byte
193         0xFF, // dummy byte
194         SSD1306_SET_SCROLL | (on ? 0x01 : 0) // Start/stop scrolling
195     };

```

```

196
197     SSD1306_send_cmd_list(cmds, count_of(cmds));
198 }
199
200 void render(uint8_t *buf, struct render_area *area) {
201     // update a portion of the display with a render area
202     uint8_t cmds[] = {
203         SSD1306_SET_COL_ADDR,
204         area->start_col,
205         area->end_col,
206         SSD1306_SET_PAGE_ADDR,
207         area->start_page,
208         area->end_page
209     };
210
211     SSD1306_send_cmd_list(cmds, count_of(cmds));
212     SSD1306_send_buf(buf, area->buflen);
213 }
214
215 static void SetPixel(uint8_t *buf, int x, int y, bool on) {
216     assert(x >= 0 && x < SSD1306_WIDTH && y >= 0 && y < SSD1306_HEIGHT);
217
218     // The calculation to determine the correct bit to set depends on which address
219     // mode we are in. This code assumes horizontal
220
221     // The video ram on the SSD1306 is split up in to 8 rows, one bit per pixel.
222     // Each row is 128 long by 8 pixels high, each byte vertically arranged, so byte 0 is x=0,
223     // y=0->7,
224     // byte 1 is x = 1, y=0->7 etc
225
226     // This code could be optimised, but is like this for clarity. The compiler
227     // should do a half decent job optimising it anyway.
228
229     const int BytesPerRow = SSD1306_WIDTH ; // x pixels, 1bpp, but each row is 8 pixel high,
230     // so (x / 8) * 8
231
232     int byte_idx = (y / 8) * BytesPerRow + x;
233     uint8_t byte = buf[byte_idx];
234
235     if (on)
236         byte |= 1 << (y % 8);
237     else
238         byte &= ~(1 << (y % 8));
239
240     buf[byte_idx] = byte;
241 }
242 // Basic Bresenham's.
243 static void DrawLine(uint8_t *buf, int x0, int y0, int x1, int y1, bool on) {
244
245     int dx = abs(x1-x0);
246     int sx = x0<x1 ? 1 : -1;
247     int dy = -abs(y1-y0);
248     int sy = y0<y1 ? 1 : -1;
249     int err = dx+dy;
250     int e2;
251
252     while (true) {
253         SetPixel(buf, x0, y0, on);
254         if (x0 == x1 && y0 == y1)
255             break;
256         e2 = 2*err;
257
258         if (e2 >= dy) {
259             err += dy;
260             x0 += sx;
261         }
262         if (e2 <= dx) {
263             err += dx;
264             y0 += sy;
265         }
266     }
267 }

```

```

258         x0 += sx;
259     }
260     if (e2 <= dx) {
261         err += dx;
262         y0 += sy;
263     }
264 }
265 }
266
267 static inline int GetFontIndex(uint8_t ch) {
268     if (ch >= 'A' && ch <= 'Z') {
269         return ch - 'A' + 1;
270     }
271     else if (ch >= '0' && ch <= '9') {
272         return ch - '0' + 27;
273     }
274     else return 0; // Not got that char so space.
275 }
276
277 static void WriteChar(uint8_t *buf, int16_t x, int16_t y, uint8_t ch) {
278     if (x > SSD1306_WIDTH - 8 || y > SSD1306_HEIGHT - 8)
279         return;
280
281     // For the moment, only write on Y row boundaries (every 8 vertical pixels)
282     y = y/8;
283
284     ch = toupper(ch);
285     int idx = GetFontIndex(ch);
286     int fb_idx = y * 128 + x;
287
288     for (int i=0;i<8;i++) {
289         buf[fb_idx++] = font[idx * 8 + i];
290     }
291 }
292
293 static void WriteString(uint8_t *buf, int16_t x, int16_t y, char *str) {
294     // Cull out any string off the screen
295     if (x > SSD1306_WIDTH - 8 || y > SSD1306_HEIGHT - 8)
296         return;
297
298     while (*str) {
299         WriteChar(buf, x, y, *str++);
300         x+=8;
301     }
302 }
303
304
305
306 #endif
307
308 int main() {
309     stdio_init_all();
310
311     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
312         !defined(PICO_DEFAULT_I2C_SCL_PIN)
313     #warning i2c / SSD1306_i2d example requires a board with I2C pins
314     puts("Default I2C pins were not defined");
315     #else
316     // useful information for picotool
317     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
318         GPIO_FUNC_I2C));
319     bi_decl(bi_program_description("SSD1306 OLED driver I2C example for the Raspberry Pi
320     Pico"));
318

```

```

319     printf("Hello, SSD1306 OLED display! Look at my raspberries..\n");
320
321     // I2C is "open drain", pull ups to keep signal high when no data is being
322     // sent
323     i2c_init(i2c_default, SSD1306_I2C_CLK * 1000);
324     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
325     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
326     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
327     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
328
329     // run through the complete initialization process
330     SSD1306_init();
331
332     // Initialize render area for entire frame (SSD1306_WIDTH pixels by SSD1306_NUM_PAGES
333     // pages)
334     struct render_area frame_area = {
335         start_col: 0,
336         end_col : SSD1306_WIDTH - 1,
337         start_page : 0,
338         end_page : SSD1306_NUM_PAGES - 1
339     };
340     calc_render_area_buf_len(&frame_area);
341
342     // zero the entire display
343     uint8_t buf[SSD1306_BUF_LEN];
344     memset(buf, 0, SSD1306_BUF_LEN);
345     render(buf, &frame_area);
346
347     // intro sequence: flash the screen 3 times
348     for (int i = 0; i < 3; i++) {
349         SSD1306_send_cmd(SSD1306_SET_ALL_ON);    // Set all pixels on
350         sleep_ms(500);
351         SSD1306_send_cmd(SSD1306_SET_ENTIRE_ON); // go back to following RAM for pixel state
352         sleep_ms(500);
353     }
354
355     // render 3 cute little raspberries
356     struct render_area area = {
357         start_page : 0,
358         end_page : (IMG_HEIGHT / SSD1306_PAGE_HEIGHT) - 1
359     };
360
361 restart:
362
363     area.start_col = 0;
364     area.end_col = IMG_WIDTH - 1;
365
366     calc_render_area_buf_len(&area);
367
368     uint8_t offset = 5 + IMG_WIDTH; // 5px padding
369
370     for (int i = 0; i < 3; i++) {
371         render(raspberry26x32, &area);
372         area.start_col += offset;
373         area.end_col += offset;
374     }
375
376     SSD1306_scroll(true);
377     sleep_ms(5000);
378     SSD1306_scroll(false);
379
380     char *text[] = {
381         "A long time ago",

```

```

382     " on an OLED ",
383     " display",
384     " far far away",
385     "Lived a small",
386     "red raspberry",
387     "by the name of",
388     " PICO"
389 };
390
391 int y = 0;
392 for (uint i = 0 ;i < count_of(text); i++) {
393     WriteString(buf, 5, y, text[i]);
394     y+=8;
395 }
396 render(buf, &frame_area);
397
398 // Test the display invert function
399 sleep_ms(3000);
400 SSD1306_send_cmd(SSD1306_SET_INV_DISP);
401 sleep_ms(3000);
402 SSD1306_send_cmd(SSD1306_SET_NORM_DISP);
403
404 bool pix = true;
405 for (int i = 0; i < 2;i++) {
406     for (int x = 0;x < SSD1306_WIDTH;x++) {
407         DrawLine(buf, x, 0, SSD1306_WIDTH - 1 - x, SSD1306_HEIGHT - 1, pix);
408         render(buf, &frame_area);
409     }
410
411     for (int y = SSD1306_HEIGHT-1; y >= 0 ;y--) {
412         DrawLine(buf, 0, y, SSD1306_WIDTH - 1, SSD1306_HEIGHT - 1 - y, pix);
413         render(buf, &frame_area);
414     }
415     pix = false;
416 }
417
418 goto restart;
419
420 #endif
421 return 0;
422 }

```

ssd1306_font.h

A simple font used in the example.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/ssd1306_i2c/ssd1306_font.h

```

1 /**
2  * Copyright (c) 2022 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 // Vertical bitmaps, A-Z, 0-9. Each is 8 pixels high and wide
8 // These are defined vertically to make them quick to copy to FB
9
10 static uint8_t font[] = {
11 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // Nothing
12 0x78, 0x14, 0x12, 0x11, 0x12, 0x14, 0x78, 0x00, //A
13 0x7f, 0x49, 0x49, 0x49, 0x49, 0x49, 0x7f, 0x00, //B
14 0x7e, 0x41, 0x41, 0x41, 0x41, 0x41, 0x41, 0x00, //C
15 0x7f, 0x41, 0x41, 0x41, 0x41, 0x41, 0x7e, 0x00, //D

```

```

16 0x7f, 0x49, 0x49, 0x49, 0x49, 0x49, 0x49, 0x00, //E
17 0x7f, 0x09, 0x09, 0x09, 0x09, 0x01, 0x01, 0x00, //F
18 0x7f, 0x41, 0x41, 0x41, 0x51, 0x51, 0x73, 0x00, //G
19 0x7f, 0x08, 0x08, 0x08, 0x08, 0x08, 0x7f, 0x00, //H
20 0x00, 0x00, 0x00, 0x00, 0x7f, 0x00, 0x00, 0x00, //I
21 0x21, 0x41, 0x41, 0x3f, 0x01, 0x01, 0x01, 0x00, //J
22 0x00, 0x7f, 0x08, 0x08, 0x14, 0x22, 0x41, 0x00, //K
23 0x7f, 0x40, 0x40, 0x40, 0x40, 0x40, 0x40, 0x00, //L
24 0x7f, 0x02, 0x04, 0x08, 0x04, 0x02, 0x7f, 0x00, //M
25 0x7f, 0x02, 0x04, 0x08, 0x10, 0x20, 0x7f, 0x00, //N
26 0x3e, 0x41, 0x41, 0x41, 0x41, 0x41, 0x3e, 0x00, //O
27 0x7f, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0e, 0x00, //P
28 0x3e, 0x41, 0x41, 0x49, 0x51, 0x61, 0x7e, 0x00, //Q
29 0x7f, 0x11, 0x11, 0x11, 0x31, 0x51, 0x0e, 0x00, //R
30 0x46, 0x49, 0x49, 0x49, 0x49, 0x30, 0x00, 0x00, //S
31 0x01, 0x01, 0x01, 0x7f, 0x01, 0x01, 0x01, 0x00, //T
32 0x3f, 0x40, 0x40, 0x40, 0x40, 0x40, 0x3f, 0x00, //U
33 0x0f, 0x10, 0x20, 0x40, 0x20, 0x10, 0x0f, 0x00, //V
34 0x7f, 0x20, 0x10, 0x08, 0x10, 0x20, 0x7f, 0x00, //W
35 0x00, 0x41, 0x22, 0x14, 0x14, 0x22, 0x41, 0x00, //X
36 0x01, 0x02, 0x04, 0x78, 0x04, 0x02, 0x01, 0x00, //Y
37 0x41, 0x61, 0x59, 0x45, 0x43, 0x41, 0x00, 0x00, //Z
38 0x3e, 0x41, 0x41, 0x49, 0x41, 0x41, 0x3e, 0x00, //0
39 0x00, 0x00, 0x42, 0x7f, 0x40, 0x00, 0x00, 0x00, //1
40 0x30, 0x49, 0x49, 0x49, 0x49, 0x46, 0x00, 0x00, //2
41 0x49, 0x49, 0x49, 0x49, 0x49, 0x49, 0x36, 0x00, //3
42 0x3f, 0x20, 0x20, 0x78, 0x20, 0x20, 0x00, 0x00, //4
43 0x4f, 0x49, 0x49, 0x49, 0x49, 0x30, 0x00, 0x00, //5
44 0x3f, 0x48, 0x48, 0x48, 0x48, 0x48, 0x30, 0x00, //6
45 0x01, 0x01, 0x01, 0x61, 0x31, 0x0d, 0x03, 0x00, //7
46 0x36, 0x49, 0x49, 0x49, 0x49, 0x49, 0x36, 0x00, //8
47 0x06, 0x09, 0x09, 0x09, 0x09, 0x09, 0x7f, 0x00, //9
48 };

```

img_to_array.py

A helper to convert an image file to an array that can be used in the example.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/ssd1306_i2c/img_to_array.py

```

1 #!/usr/bin/env python3
2
3 # Converts a grayscale image into a format able to be
4 # displayed by the SSD1306 driver in horizontal addressing mode
5
6 # usage: python3 img_to_array.py <logo.bmp>
7
8 # depends on the Pillow library
9 # `python3 -m pip install --upgrade Pillow`
10
11 from PIL import Image
12 import sys
13 from pathlib import Path
14
15 OLED_HEIGHT = 32
16 OLED_WIDTH = 128
17 OLED_PAGE_HEIGHT = 8
18
19 if len(sys.argv) < 2:
20     print("No image path provided.")
21     sys.exit()
22
23 img_path = sys.argv[1]

```



```

24
25 try:
26     im = Image.open(img_path)
27 except OSError:
28     raise Exception("Oops! The image could not be opened.")
29
30 img_width = im.size[0]
31 img_height = im.size[1]
32
33 if img_width > OLED_WIDTH or img_height > OLED_HEIGHT:
34     print(f'Your image is {img_width} pixels wide and {img_height} pixels high, but...')
35     raise Exception(f"OLED display only {OLED_WIDTH} pixels wide and {OLED_HEIGHT} pixels
    high!")
36
37 if not (im.mode == "1" or im.mode == "L"):
38     raise Exception("Image must be grayscale only")
39
40 # black or white
41 out = im.convert("1")
42
43 img_name = Path(im.filename).stem
44
45 # `pixels` is a flattened array with the top left pixel at index 0
46 # and bottom right pixel at the width*height-1
47 pixels = list(out.getdata())
48
49 # swap white for black and swap (255, 0) for (1, 0)
50 pixels = [0 if x == 255 else 1 for x in pixels]
51
52 # our goal is to divide the image into 8-pixel high pages
53 # and turn a pixel column into one byte, eg for one page:
54 # 0 1 0 ....
55 # 1 0 0
56 # 1 1 1
57 # 0 0 1
58 # 1 1 0
59 # 0 1 0
60 # 1 1 1
61 # 0 0 1 ....
62
63 # we get 0x6A, 0xAE, 0x33 ... and so on
64 # as `pixels` is flattened, each bit in a column is IMG_WIDTH apart from the next
65
66 buffer = []
67 for i in range(img_height // OLED_PAGE_HEIGHT):
68     start_index = i*img_width*OLED_PAGE_HEIGHT
69     for j in range(img_width):
70         out_byte = 0
71         for k in range(OLED_PAGE_HEIGHT):
72             out_byte |= pixels[k*img_width + start_index + j] << k
73         buffer.append(f'{out_byte:#04x}')
74
75 buffer = ", ".join(buffer)
76 buffer_hex = f'static uint8_t {img_name}[] = {{{buffer}}};\n'
77
78 with open(f'{img_name}.h', 'wt') as file:
79     file.write(f'#define IMG_WIDTH {img_width}\n')
80     file.write(f'#define IMG_HEIGHT {img_height}\n\n')
81     file.write(buffer_hex)

```

raspberrypi26x32.bmp

Example image file of a Raspberry.

raspberrypi26x32.h

The example image file converted to an C array.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/ssd1306_i2c/raspberry26x32.h

```
1 #define IMG_WIDTH 26
2 #define IMG_HEIGHT 32
3
4 static uint8_t raspberry26x32[] = { 0x0, 0x0, 0xe, 0x7e, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xfe, 0xfe, 0xfc, 0xf8, 0xfc, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0x7e, 0x1e, 0x0,
    0x0, 0x0, 0x80, 0xe0, 0xf8, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xfd, 0xf8, 0xe0, 0x80, 0x0, 0x0, 0x1e, 0x7f, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0x7f, 0x1e, 0x0, 0x0, 0x0, 0x3, 0x7, 0xf, 0x1f, 0x3f, 0x3f, 0x7f,
    0xff, 0xff, 0xff, 0xff, 0x7f, 0x7f, 0x3f, 0x3f, 0x1f, 0x1f, 0xf, 0x7, 0x3, 0x0, 0x0};
```

Bill of Materials

Table 49. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
SSD1306-based OLED display	1	Adafruit part
M/M Jumper wires	4	generic part

Attaching a PA1010D Mini GPS module via I2C

This example code shows how to interface the Raspberry Pi Pico to the PA1010D Mini GPS module

This allows you to read basic location and time data from the Recommended Minimum Specific GNSS Sentence (GNRMC protocol) and displays it in a user-friendly format. The datasheet for the module can be found on https://cdn-learn.adafruit.com/assets/assets/000/084/295/original/CD_PA1010D_Datasheet_v.03.pdf?1573833002. The output sentence is read and parsed to split the data fields into a 2D character array, which are then individually printed out. The commands to use different protocols and change settings are found on https://www.sparkfun.com/datasheets/GPS/Modules/PMTK_Protocol.pdf. Additional protocols can be used by editing the `init_command` array.

NOTE

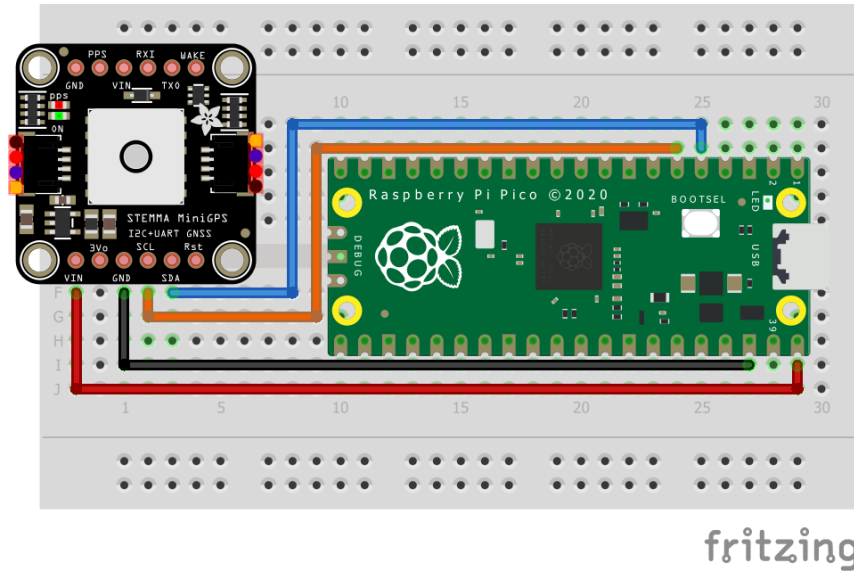
Each command requires a checksum after the asterisk. The checksum can be calculated for your command using the following website: <https://nmeachecksum.eqth.net/>.

The GPS needs to be used outdoors in open skies and requires about 15 seconds to acquire a satellite signal in order to display valid data. When the signal is detected, the device will blink a green LED at 1 Hz.

Wiring information

Wiring up the device requires 4 jumpers, to connect VDD, GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 3V pin.

Figure 25. Wiring
Diagram for PA1010D.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/pa1010d_i2c/CMakeLists.txt

```
1 add_executable(pa1010d_i2c
2     pa1010d_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(pa1010d_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(pa1010d_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(pa1010d_i2c)
```

pa1010d_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/pa1010d_i2c/pa1010d_i2c.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12 #include "string.h"
13
```

```

14 /* Example code to talk to a PA1010D Mini GPS module.
15
16 This example reads the Recommended Minimum Specific GNSS Sentence, which includes basic
location and time data, each second, formats and displays it.
17
18 Connections on Raspberry Pi Pico board, other boards may vary.
19
20 GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is 4 (physical pin 6)) -> SDA on PA1010D board
21 GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is 5 (physical pin 7)) -> SCL on PA1010D board
22 3.3v (physical pin 36) -> VCC on PA1010D board
23 GND (physical pin 38) -> GND on PA1010D board
24 */
25
26 const int addr = 0x10;
27 #define MAX_READ 250
28
29 #ifdef i2c_default
30
31 void pa1010d_write_command(const char command[], int com_length) {
32     // Convert character array to bytes for writing
33     uint8_t int_command[com_length];
34
35     for (int i = 0; i < com_length; ++i) {
36         int_command[i] = command[i];
37         i2c_write_blocking(i2c_default, addr, &int_command[i], 1, true);
38     }
39 }
40
41 void pa1010d_parse_string(char output[], char protocol[]) {
42     // Finds location of protocol message in output
43     char *com_index = strstr(output, protocol);
44     int p = com_index - output;
45
46     // Splits components of output sentence into array
47     #define NO_OF_FIELDS 14
48     #define MAX_LEN 15
49
50     int n = 0;
51     int m = 0;
52
53     char gps_data[NO_OF_FIELDS][MAX_LEN];
54     memset(gps_data, 0, sizeof(gps_data));
55
56     bool complete = false;
57     while (output[p] != '$' && n < MAX_LEN && complete == false) {
58         if (output[p] == ',' || output[p] == '*') {
59             n += 1;
60             m = 0;
61         } else {
62             gps_data[n][m] = output[p];
63             // Checks if sentence is complete
64             if (m < NO_OF_FIELDS) {
65                 m++;
66             } else {
67                 complete = true;
68             }
69         }
70         p++;
71     }
72
73     // Displays GNRMC data
74     // Similarly, additional if statements can be used to add more protocols
75     if (strcmp(protocol, "GNRMC") == 0) {
76         printf("Protocol:%s\n", gps_data[0]);

```

```

77     printf("UTC Time: %s\n", gps_data[1]);
78     printf("Status: %s\n", gps_data[2][0] == 'V' ? "Data invalid. GPS fix not found." :
    "Data Valid");
79     printf("Latitude: %s\n", gps_data[3]);
80     printf("N/S indicator: %s\n", gps_data[4]);
81     printf("Longitude: %s\n", gps_data[5]);
82     printf("E/W indicator: %s\n", gps_data[6]);
83     printf("Speed over ground: %s\n", gps_data[7]);
84     printf("Course over ground: %s\n", gps_data[8]);
85     printf("Date: %c%c/%c%c/%c%c\n", gps_data[9][0], gps_data[9][1], gps_data[9][2],
    gps_data[9][3], gps_data[9][4],
86         gps_data[9][5]);
87     printf("Magnetic Variation: %s\n", gps_data[10]);
88     printf("E/W degree indicator: %s\n", gps_data[11]);
89     printf("Mode: %s\n", gps_data[12]);
90     printf("Checksum: %c%c\n", gps_data[13][0], gps_data[13][1]);
91 }
92 }
93
94 void pa1010d_read_raw(char numcommand[]) {
95     uint8_t buffer[MAX_READ];
96
97     int i = 0;
98     bool complete = false;
99
100     i2c_read_blocking(i2c_default, addr, buffer, MAX_READ, false);
101
102     // Convert bytes to characters
103     while (i < MAX_READ && complete == false) {
104         numcommand[i] = buffer[i];
105         // Stop converting at end of message
106         if (buffer[i] == 10 && buffer[i + 1] == 10) {
107             complete = true;
108         }
109         i++;
110     }
111 }
112
113 #endif
114
115 int main() {
116     stdio_init_all();
117     #if !defined(i2c_default) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
    !defined(PICO_DEFAULT_I2C_SCL_PIN)
118     #warning i2c/mpu6050_i2c example requires a board with I2C pins
119     puts("Default I2C pins were not defined");
120 #else
121
122     char numcommand[MAX_READ];
123
124     // Decide which protocols you would like to retrieve data from
125     char init_command[] = "$PMTK314,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0*29\r\n";
126
127     // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
128     i2c_init(i2c_default, 400 * 1000);
129     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
130     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
131     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
132     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
133
134     // Make the I2C pins available to picotool
135     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
    GPIO_FUNC_I2C));
136

```

```
137     printf("Hello, PA1010D! Reading raw data from module...\n");
138
139     pa1010d_write_command(init_command, sizeof(init_command));
140
141     while (1) {
142         // Clear array
143         memset(numcommand, 0, MAX_READ);
144         // Read and re-format
145         pa1010d_read_raw(numcommand);
146         pa1010d_parse_string(numcommand, "GNRMC");
147
148         // Wait for data to refresh
149         sleep_ms(1000);
150
151         // Clear terminal
152         printf("\033[1;1H\033[2J");
153     }
154 #endif
155 }
```

Bill of Materials

Table 50. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
PA1010D board	1	https://shop.pimoroni.com/products/pa1010d-gps-breakout
M/M Jumper wires	4	generic part

Attaching a PCF8523 Real Time Clock via I2C

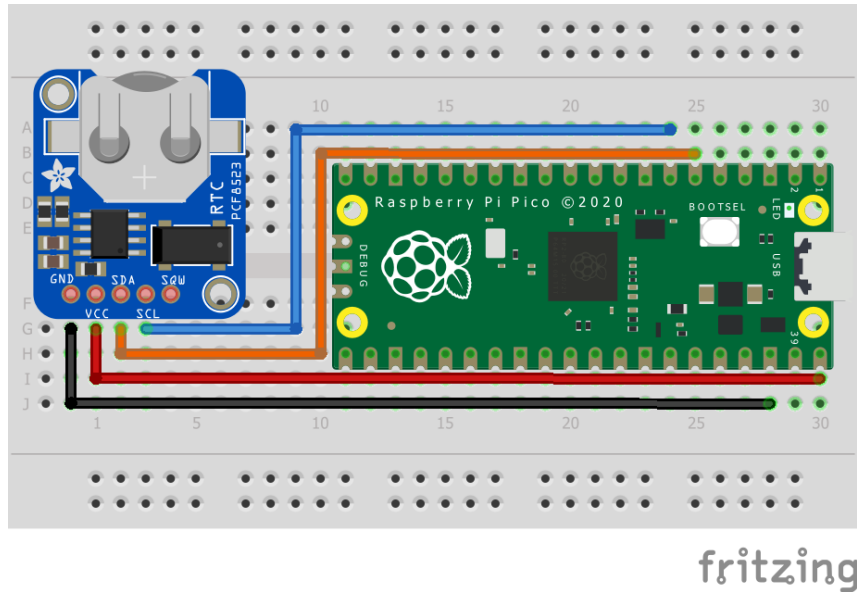
This example code shows how to interface the Raspberry Pi Pico to the PCF8523 Real Time Clock

This example allows you to initialise the current time and date and then displays it every half-second. Additionally it lets you set an alarm for a particular time and date and raises an alert accordingly. More information about the module is available at <https://learn.adafruit.com/adafruit-pcf8523-real-time-clock>.

Wiring information

Wiring up the device requires 4 jumpers, to connect VDD, GND, SDA and SCL. The example here uses I2C port 0, which is assigned to GPIO 4 (SDA) and 5 (SCL) in software. Power is supplied from the 5V pin.

Figure 26. Wiring
Diagram for PCF8523.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/pcf8523_i2c/CMakeLists.txt

```
1 add_executable(pcf8523_i2c
2     pcf8523_i2c.c
3 )
4
5 # pull in common dependencies and additional i2c hardware support
6 target_link_libraries(pcf8523_i2c pico_stdlib hardware_i2c)
7
8 # create map/bin/hex file etc.
9 pico_add_extra_outputs(pcf8523_i2c)
10
11 # add url via pico_set_program_url
12 example_auto_set_url(pcf8523_i2c)
```

pcf8523_i2c.c

The example code.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/i2c/pcf8523_i2c/pcf8523_i2c.c

```
1 /**
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include <string.h>
9 #include "pico/stdlib.h"
10 #include "pico/binary_info.h"
11 #include "hardware/i2c.h"
12
13 /* Example code to talk to a PCF8520 Real Time Clock module
```

```

14
15     Connections on Raspberry Pi Pico board, other boards may vary.
16
17     GPIO PICO_DEFAULT_I2C_SDA_PIN (On Pico this is 4 (physical pin 6)) -> SDA on PCF8520 board
18     GPIO PICO_DEFAULT_I2C_SCK_PIN (On Pico this is 5 (physical pin 7)) -> SCL on PCF8520 board
19     5V (physical pin 40) -> VCC on PCF8520 board
20     GND (physical pin 38) -> GND on PCF8520 board
21 */
22
23 #ifdef i2c_default
24
25 // By default these devices are on bus address 0x68
26 static int addr = 0x68;
27
28 static void pcf8520_reset() {
29     // Two byte reset. First byte register, second byte data
30     // There are a load more options to set up the device in different ways that could be
31     // added here
32     uint8_t buf[] = {0x00, 0x58};
33     i2c_write_blocking(i2c_default, addr, buf, 2, false);
34 }
35
36 static void pcf820_write_current_time() {
37     // buf[0] is the register to write to
38     // buf[1] is the value that will be written to the register
39     uint8_t buf[2];
40
41     //Write values for the current time in the array
42     //index 0 -> second: bits 4-6 are responsible for the ten's digit and bits 0-3 for the
43     //unit's digit
44     //index 1 -> minute: bits 4-6 are responsible for the ten's digit and bits 0-3 for the
45     //unit's digit
46     //index 2 -> hour: bits 4-5 are responsible for the ten's digit and bits 0-3 for the
47     //unit's digit
48     //index 3 -> day of the month: bits 4-5 are responsible for the ten's digit and bits 0-3
49     //for the unit's digit
50     //index 4 -> day of the week: where Sunday = 0x00, Monday = 0x01, Tuesday... ..Saturday =
51     //0x06
52     //index 5 -> month: bit 4 is responsible for the ten's digit and bits 0-3 for the unit's
53     //digit
54     //index 6 -> year: bits 4-7 are responsible for the ten's digit and bits 0-3 for the
55     //unit's digit
56
57     //NOTE: if the value in the year register is a multiple for 4, it will be considered a
58     //leap year and hence will include the 29th of February
59
60     uint8_t current_val[7] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
61
62     for (int i = 3; i < 10; ++i) {
63         buf[0] = i;
64         buf[1] = current_val[i - 3];
65         i2c_write_blocking(i2c_default, addr, buf, 2, false);
66     }
67 }
68
69 static void pcf8520_read_raw(uint8_t *buffer) {
70     // For this particular device, we send the device the register we want to read
71     // first, then subsequently read from the device. The register is auto incrementing
72     // so we don't need to keep sending the register we want, just the first.
73
74     // Start reading acceleration registers from register 0x3B for 6 bytes
75     uint8_t val = 0x03;
76     i2c_write_blocking(i2c_default, addr, &val, 1, true); // true to keep master control of
77     bus

```



```

68     i2c_read_blocking(i2c_default, addr, buffer, 7, false);
69 }
70
71
72 void pcf8520_set_alarm() {
73     // buf[0] is the register to write to
74     // buf[1] is the value that will be written to the register
75     uint8_t buf[2];
76
77     // Default value of alarm register is 0x80
78     // Set bit 8 of values to 0 to activate that particular alarm
79     // Index 0 -> minute: bits 4-5 are responsible for the ten's digit and bits 0-3 for the
    unit's digit
80     // Index 1 -> hour: bits 4-6 are responsible for the ten's digit and bits 0-3 for the
    unit's digit
81     // Index 2 -> day of the month: bits 4-5 are responsible for the ten's digit and bits 0-3
    for the unit's digit
82     // Index 3 -> day of the week: where Sunday = 0x00, Monday = 0x01, Tuesday... ..Saturday
    = 0x06
83
84     uint8_t alarm_val[4] = {0x01, 0x80, 0x80, 0x80};
85     // Write alarm values to registers
86     for (int i = 10; i < 14; ++i) {
87         buf[0] = (uint8_t) i;
88         buf[1] = alarm_val[i - 10];
89         i2c_write_blocking(i2c_default, addr, buf, 2, false);
90     }
91 }
92
93 void pcf8520_check_alarm() {
94     // Check bit 3 of control register 2 for alarm flags
95     uint8_t status[1];
96     uint8_t val = 0x01;
97     i2c_write_blocking(i2c_default, addr, &val, 1, true); // true to keep master control of
    bus
98     i2c_read_blocking(i2c_default, addr, status, 1, false);
99
100     if ((status[0] & 0x08) == 0x08) {
101         printf("ALARM RINGING");
102     } else {
103         printf("Alarm not triggered yet");
104     }
105 }
106
107
108 void pcf8520_convert_time(int conv_time[7], const uint8_t raw_time[7]) {
109     // Convert raw data into time
110     conv_time[0] = (10 * (int) ((raw_time[0] & 0x70) >> 4)) + ((int) (raw_time[0] & 0x0F));
111     conv_time[1] = (10 * (int) ((raw_time[1] & 0x70) >> 4)) + ((int) (raw_time[1] & 0x0F));
112     conv_time[2] = (10 * (int) ((raw_time[2] & 0x30) >> 4)) + ((int) (raw_time[2] & 0x0F));
113     conv_time[3] = (10 * (int) ((raw_time[3] & 0x30) >> 4)) + ((int) (raw_time[3] & 0x0F));
114     conv_time[4] = (int) (raw_time[4] & 0x07);
115     conv_time[5] = (10 * (int) ((raw_time[5] & 0x10) >> 4)) + ((int) (raw_time[5] & 0x0F));
116     conv_time[6] = (10 * (int) ((raw_time[6] & 0xF0) >> 4)) + ((int) (raw_time[6] & 0x0F));
117 }
118 #endif
119
120 int main() {
121     stdio_init_all();
122     #if !defined(I2C_DEFAULT) || !defined(PICO_DEFAULT_I2C_SDA_PIN) ||
    !defined(PICO_DEFAULT_I2C_SCL_PIN)
123     #warning i2c/pcf8520_i2c example requires a board with I2C pins
124     puts("Default I2C pins were not defined");
125 #else

```

```
126     printf("Hello, PCF8520! Reading raw data from registers...\n");
127
128     // This example will use I2C0 on the default SDA and SCL pins (4, 5 on a Pico)
129     i2c_init(i2c_default, 400 * 1000);
130     gpio_set_function(PICO_DEFAULT_I2C_SDA_PIN, GPIO_FUNC_I2C);
131     gpio_set_function(PICO_DEFAULT_I2C_SCL_PIN, GPIO_FUNC_I2C);
132     gpio_pull_up(PICO_DEFAULT_I2C_SDA_PIN);
133     gpio_pull_up(PICO_DEFAULT_I2C_SCL_PIN);
134     // Make the I2C pins available to picotool
135     bi_decl(bi_2pins_with_func(PICO_DEFAULT_I2C_SDA_PIN, PICO_DEFAULT_I2C_SCL_PIN,
GPIO_FUNC_I2C));
136
137     pcf8520_reset();
138
139     pcf820_write_current_time();
140     pcf8520_set_alarm();
141     pcf8520_check_alarm();
142
143     uint8_t raw_time[7];
144     int real_time[7];
145     char days_of_week[7][12] = {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday"};
146
147     while (1) {
148
149         pcf8520_read_raw(raw_time);
150         pcf8520_convert_time(real_time, raw_time);
151
152         printf("Time: %02d : %02d : %02d\n", real_time[2], real_time[1], real_time[0]);
153         printf("Date: %s %02d / %02d / %02d\n", days_of_week[real_time[4]], real_time[3],
real_time[5], real_time[6]);
154         pcf8520_check_alarm();
155
156         sleep_ms(500);
157
158         // Clear terminal
159         printf("\033[1;1H\033[2J");
160     }
161 #endif
162 }
```

Bill of Materials

Table 51. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
PCF8523 board	1	https://www.adafruit.com/product/3295
M/M Jumper wires	4	generic part

Interfacing 1-Wire devices to the Pico

This example demonstrates how to use 1-Wire devices with the Raspberry Pi Pico (RP2040). 1-Wire is an interface that

enables a master to control several slave devices over a simple shared serial bus.

The example provides a 1-Wire library that is used to take readings from a set of connected DS18B20 1-Wire temperature sensors. The results are sent to the default serial terminal connected via USB or UART as configured in the SDK.

The library uses a driver based on the RP2040 PIO state machine to generate accurate bus timings and control the 1-Wire bus via a GPIO pin.

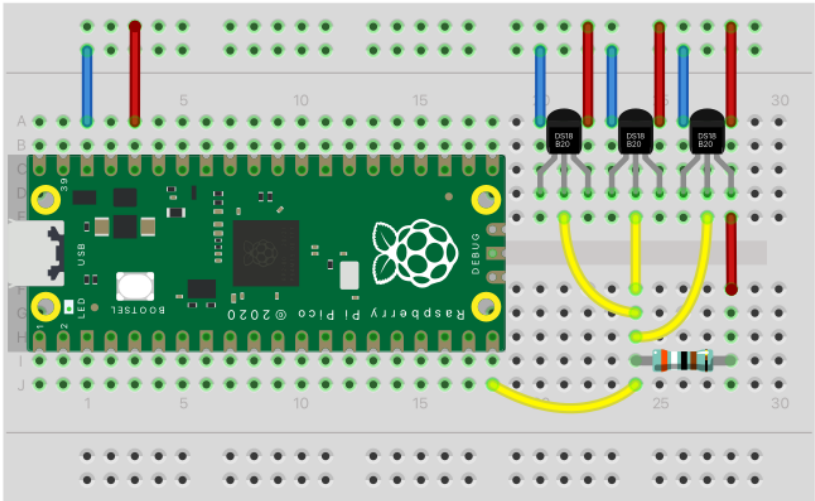
1-Wire® is a registered trademark of Maxim Integrated Products, Inc.

Wiring information

Connect one or more DS18B20 sensors to the Pico as shown in the diagram and table below.

Connect GPIO 15 to 3V3(OUT) with a pull-up resistor of about 4k ohms.

Figure 27. Wiring diagram



fritzing

Table 52. Connections table

Pico	pin	DS18B20	pin / sensor wire
GND	38 or equivalent	GND	1 / Black
GPIO 15	20	DQ	2 / Yellow
3V3(OUT)	36	VDD	3 / Red

Bill of materials

Table 53. A list of materials for the example circuit

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	1	https://www.raspberrypi.com/products/raspberry-pi-pico/
DS18B20	3	chip or wired sensor
3900 ohm resistor	1	generic part
M/M jumper wire	13	generic part

List of files

CMakeLists.txt

CMake file to incorporate the example in the build tree.

Pico Examples: <https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/CMakeLists.txt>

```

1 add_executable(pio_onewire)
2
3 target_sources(pio_onewire PRIVATE onewire.c)
4
5 add_subdirectory(onewire_library)
6
7 target_link_libraries(pio_onewire PRIVATE
8     pico_stdlib
9     hardware_pio
10    onewire_library)
11
12 pico_add_extra_outputs(pio_onewire)
13
14 # add url via pico_set_program_url
15 example_auto_set_url(pio_onewire)

```

onewire.c

Source code for the example program.

Pico Examples: <https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/onewire.c>

```

1 /**
2  * Copyright (c) 2023 mjcross
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "pico/binary_info.h"
10
11 #include "onewire_library.h" // onewire library functions
12 #include "ow_rom.h"         // onewire ROM command codes
13 #include "ds18b20.h"        // ds18b20 function codes
14
15 // Demonstrates the PIO onewire driver by taking readings from a set of
16 // ds18b20 1-wire temperature sensors.
17
18 int main() {
19     stdio_init_all();
20
21     PIO pio = pio0;
22     uint gpio = 15;
23
24     OW ow;
25     uint offset;
26     // add the program to the PIO shared address space
27     if (pio_can_add_program (pio, &onewire_program)) {
28         offset = pio_add_program (pio, &onewire_program);
29
30         // claim a state machine and initialise a driver instance
31         if (ow_init (&ow, pio, offset, gpio)) {
32

```

```

33         // find and display 64-bit device addresses
34         int maxdevs = 10;
35         uint64_t romcode[maxdevs];
36         int num_devs = ow_romsearch (&ow, romcode, maxdevs, OW_SEARCH_ROM);
37
38         printf("Found %d devices\n", num_devs);
39         for (int i = 0; i < num_devs; i += 1) {
40             printf("\t%d: 0x%llx\n", i, romcode[i]);
41         }
42         putchar ('\n');
43
44         while (num_devs > 0) {
45             // start temperature conversion in parallel on all devices
46             // (see ds18b20 datasheet)
47             ow_reset (&ow);
48             ow_send (&ow, OW_SKIP_ROM);
49             ow_send (&ow, DS18B20_CONVERT_T);
50
51             // wait for the conversions to finish
52             while (ow_read(&ow) == 0);
53
54             // read the result from each device
55             for (int i = 0; i < num_devs; i += 1) {
56                 ow_reset (&ow);
57                 ow_send (&ow, OW_MATCH_ROM);
58                 for (int b = 0; b < 64; b += 8) {
59                     ow_send (&ow, romcode[i] >> b);
60                 }
61                 ow_send (&ow, DS18B20_READ_SCRATCHPAD);
62                 int16_t temp = 0;
63                 temp = ow_read (&ow) | (ow_read (&ow) << 8);
64                 printf ("\t%d: %f", i, temp / 16.0);
65             }
66             putchar ('\n');
67         }
68
69     } else {
70         puts ("could not initialise the driver");
71     }
72 } else {
73     puts ("could not add the program");
74 }
75
76 while(true);
77 }

```

ow_rom.h

Header file with generic ROM command codes for 1-Wire devices.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/ow_rom.h

```

1 // Generic ROM commands for 1-Wire devices
2 // https://www.analog.com/en/technical-articles/guide-to-1wire-communication.html
3 //
4 #define OW_READ_ROM          0x33
5 #define OW_MATCH_ROM         0x55
6 #define OW_SKIP_ROM          0xCC
7 #define OW_ALARM_SEARCH      0xEC
8 #define OW_SEARCH_ROM         0xF0

```

ds18b20.h

Header file with function command codes for the DS18B20 device.

Pico Examples: <https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/ds18b20.h>

```
1 // Function commands for d218b20 1-Wire temperature sensor
2 // https://www.analog.com/en/products/ds18b20.html
3 //
4 #define DS18B20_CONVERT_T      0x44
5 #define DS18B20_WRITE_SCRATCHPAD 0x4e
6 #define DS18B20_READ_SCRATCHPAD 0xbe
7 #define DS18B20_COPY_SCRATCHPAD 0x48
8 #define DS18B20_RECALL_EE      0xb8
9 #define DS18B20_READ_POWER_SUPPLY 0xb4
```

onewire_library/

Subdirectory containing the 1-Wire library and driver.

onewire_library/CMakeLists.txt

CMake file to build the 1-Wire library and driver.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/onewire_library/CMakeLists.txt

```
1 add_library(onewire_library INTERFACE)
2 target_sources(onewire_library INTERFACE ${CMAKE_CURRENT_SOURCE_DIR}/onewire_library.c)
3
4 # invoke pio_asm to assemble the state machine programs
5 #
6 pico_generate_pio_header(onewire_library ${CMAKE_CURRENT_LIST_DIR}/onewire_library.pio)
7
8 target_link_libraries(onewire_library INTERFACE
9     pico_stdlib
10    hardware_pio
11    )
12
13 # add the `binary` directory so that the generated headers are included in the project
14 #
15 target_include_directories(onewire_library INTERFACE
16     ${CMAKE_CURRENT_SOURCE_DIR}
17     ${CMAKE_CURRENT_BINARY_DIR}
18    )
```

onewire_library/onewire_library.c

Source code for the 1-Wire user functions.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/onewire_library/onewire_library.c

```
1 /**
2  * Copyright (c) 2023 mjcross
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7 #include "pico/stdlib.h"
8 #include "hardware/gpio.h"
9 #include "hardware/pio.h"
10
11 #include "onewire_library.h"
12
```

```

13
14 // Create a driver instance and populate the provided OW structure.
15 // Returns: True on success.
16 // ow: A pointer to a blank OW structure to hold the driver parameters.
17 // pio: The PIO hardware instance to use.
18 // offset: The location of the onewire program in the PIO shared address space.
19 // gpio: The pin to use for the bus (NB: see the README).
20 bool ow_init (OW *ow, PIO pio, uint offset, uint gpio) {
21     int sm = pio_claim_unused_sm (pio, false);
22     if (sm == -1) {
23         return false;
24     }
25     gpio_init (gpio);           // enable the gpio and clear any output value
26     pio_gpio_init (pio, gpio);  // set the function to PIO output
27     ow->gpio = gpio;
28     ow->pio = pio;
29     ow->offset = offset;
30     ow->sm = (uint)sm;
31     ow->jmp_reset = onewire_reset_instr (ow->offset); // assemble the bus reset instruction
32     onewire_sm_init (ow->pio, ow->sm, ow->offset, ow->gpio, 8); // set 8 bits per word
33     return true;
34 }
35
36
37 // Send a binary word on the bus (LSB first).
38 // ow: A pointer to an OW driver struct.
39 // data: The word to be sent.
40 void ow_send (OW *ow, uint data) {
41     pio_sm_put_blocking (ow->pio, ow->sm, (uint32_t)data);
42     pio_sm_get_blocking (ow->pio, ow->sm); // discard the response
43 }
44
45
46 // Read a binary word from the bus.
47 // Returns: the word read (LSB first).
48 // ow: pointer to an OW driver struct
49 uint8_t ow_read (OW *ow) {
50     pio_sm_put_blocking (ow->pio, ow->sm, 0xff); // generate read slots
51     return (uint8_t)(pio_sm_get_blocking (ow->pio, ow->sm) >> 24); // shift response into
    bits 0..7
52 }
53
54
55 // Reset the bus and detect any connected slaves.
56 // Returns: true if any slaves responded.
57 // ow: pointer to an OW driver struct
58 bool ow_reset (OW *ow) {
59     pio_sm_exec_wait_blocking (ow->pio, ow->sm, ow->jmp_reset);
60     if ((pio_sm_get_blocking (ow->pio, ow->sm) & 1) == 0) { // apply pin mask (see pio
        program)
61         return true; // a slave pulled the bus low
62     }
63     return false;
64 }
65
66
67 // Find ROM codes (64-bit hardware addresses) of all connected devices.
68 // See https://www.analog.com/en/app-notes/1wire-search-algorithm.html
69 // Returns: the number of devices found (up to maxdevs) or -1 if an error occurred.
70 // ow: pointer to an OW driver struct
71 // romcodes: location at which store the addresses (NULL means don't store)
72 // maxdevs: maximum number of devices to find (0 means no limit)
73 // command: 1-Wire search command (e.g. OW_SEARCHROM or OW_ALARM_SEARCH)
74 int ow_romsearch (OW *ow, uint64_t *romcodes, int maxdevs, uint command) {

```

```

75     int index;
76     uint64_t romcode = 0ull;
77     int branch_point;
78     int next_branch_point = -1;
79     int num_found = 0;
80     bool finished = false;
81
82     onewire_sm_init (ow->pio, ow->sm, ow->offset, ow->gpio, 1); // set driver to 1-bit mode
83
84     while (finished == false && (maxdevs == 0 || num_found < maxdevs)) {
85         finished = true;
86         branch_point = next_branch_point;
87         if (ow_reset (ow) == false) {
88             num_found = 0; // no slaves present
89             finished = true;
90             break;
91         }
92         for (int i = 0; i < 8; i += 1) { // send search command as single bits
93             ow_send (ow, command >> i);
94         }
95         for (index = 0; index < 64; index += 1) { // determine romcode bits 0..63 (see ref)
96             uint a = ow_read (ow);
97             uint b = ow_read (ow);
98             if (a == 0 && b == 0) { // (a, b) = (0, 0)
99                 if (index == branch_point) {
100                     ow_send (ow, 1);
101                     romcode |= (1ull << index);
102                 } else {
103                     if (index > branch_point || (romcode & (1ull << index)) == 0) {
104                         ow_send(ow, 0);
105                         finished = false;
106                         romcode &= ~(1ull << index);
107                         next_branch_point = index;
108                     } else { // index < branch_point or romcode[index] = 1
109                         ow_send (ow, 1);
110                     }
111                 }
112             } else if (a != 0 && b != 0) { // (a, b) = (1, 1) error (e.g. device
disconnected)
113                 num_found = -2; // function will return -1
114                 finished = true;
115                 break; // terminate for loop
116             } else {
117                 if (a == 0) { // (a, b) = (0, 1) or (1, 0)
118                     ow_send (ow, 0);
119                     romcode &= ~(1ull << index);
120                 } else {
121                     ow_send (ow, 1);
122                     romcode |= (1ull << index);
123                 }
124             }
125         } // end of for loop
126
127         if (romcodes != NULL) {
128             romcodes[num_found] = romcode; // store the romcode
129         }
130         num_found += 1;
131     } // end of while loop
132
133     onewire_sm_init (ow->pio, ow->sm, ow->offset, ow->gpio, 8); // restore 8-bit mode
134     return num_found;
135 }

```


onewire_library/onewire_library.h

Header file for the 1-Wire user functions and types.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/onewire_library/onewire_library.h

```

1 #ifndef _ONEWIRE_LIBRARY_H
2 #define _ONEWIRE_LIBRARY_H
3
4 #include "hardware/pio.h"
5 #include "hardware/clocks.h" // for clock_get_hz() in generated header
6 #include "onewire_library.pio.h" // generated by pioasm
7
8 typedef struct {
9     PIO pio;
10    uint sm;
11    uint jmp_reset;
12    int offset;
13    int gpio;
14 } OW;
15
16 bool ow_init (OW *ow, PIO pio, uint offset, uint gpio);
17 void ow_send (OW *ow, uint data);
18 uint8_t ow_read (OW *ow);
19 bool ow_reset (OW *ow);
20 int ow_romsearch (OW *ow, uint64_t *romcodes, int maxdevs, uint command);
21
22 #endif

```

onewire_library/onewire_library.pio

PIO assembly code for the 1-Wire driver.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/onewire/onewire_library/onewire_library.pio

```

1 ;
2 ; Copyright (c) 2023 mjcross
3 ;
4 ; SPDX-License-Identifier: BSD-3-Clause
5 ;
6 ;
7 ; Implements a Maxim 1-Wire bus with a GPIO pin.
8 ;
9 ; Place data words to be transmitted in the TX FIFO and read the results from the
10 ; RX FIFO. To reset the bus execute a jump to 'reset_bus' using the opcode from
11 ; the provided function.
12 ;
13 ; At 1us per cycle as initialised below the timings are those recommended by:
14 ; https://www.analog.com/en/technical-articles/1wire-communication-through-software.html
15 ;
16 ; Notes:
17 ; (1) The code will stall with the bus in a safe state if the FIFOs are empty/full.
18 ; (2) The bus must be pulled up with an external pull-up resistor of about 4k.
19 ;     The internal GPIO resistors are too high (~50k) to work reliably for this.
20 ; (3) Do not connect the GPIO pin directly to a bus powered at more than 3.3V.
21 ;
22 .program onewire
23 .side_set 1 pindirs
24
25 PUBLIC reset_bus:
26     set x, 28        side 1 [15]    ; pull bus low                16
27 loop_a: jmp x-- loop_a side 1 [15]    ;                          29 x 16
28     set x, 8         side 0 [6]    ; release bus                7

```

```

29 loop_b: jmp x-- loop_b side 0 [6] ; 9 x 7
30
31 mov isr, pins side 0 ; read all pins to ISR (avoids autopush) 1
32 push side 0 ; push result manually 1
33 set x, 24 side 0 [7] ; 8
34 loop_c: jmp x-- loop_c side 0 [15] ; 25 x 16
35
36 .wrap_target
37 PUBLIC fetch_bit:
38 out x, 1 side 0 ; shift next bit from OSR (autopull) 1
39 jmp !x send_0 side 1 [5] ; pull bus low, branch if sending '0' 6
40
41 send_1: ; send a '1' bit
42 set x, 2 side 0 [8] ; release bus, wait for slave response 9
43 in pins, 1 side 0 [4] ; read bus, shift bit to ISR (autopush) 5
44 loop_e: jmp x-- loop_e side 0 [15] ; 3 x 16
45 jmp fetch_bit side 0 ; 1
46
47 send_0: ; send a '0' bit
48 set x, 2 side 1 [5] ; continue pulling bus low 6
49 loop_d: jmp x-- loop_d side 1 [15] ; 3 x 16
50 in null, 1 side 0 [8] ; release bus, shift 0 to ISR (autopush) 9
51 .wrap
52 ;; (17 instructions)
53
54
55 % c-sdk {
56 static inline void onewire_sm_init (PIO pio, uint sm, uint offset, uint pin_num, uint
bits_per_word) {
57
58 // create a new state machine configuration
59 pio_sm_config c = onewire_program_get_default_config (offset);
60
61 // Input Shift Register configuration settings
62 sm_config_set_in_shift (
63 &c,
64 true, // shift direction: right
65 true, // autopush: enabled
66 bits_per_word // autopush threshold
67 );
68
69 // Output Shift Register configuration settings
70 sm_config_set_out_shift (
71 &c,
72 true, // shift direction: right
73 true, // autopull: enabled
74 bits_per_word // autopull threshold
75 );
76
77 // configure the input and sideset pin groups to start at `pin_num`
78 sm_config_set_in_pins (&c, pin_num);
79 sm_config_set_sideset_pins (&c, pin_num);
80
81 // configure the clock divider for 1 usec per instruction
82 float div = clock_get_hz (clk_sys) * 1e-6;
83 sm_config_set_clkdiv (&c, div);
84
85 // apply the configuration and initialise the program counter
86 pio_sm_init (pio, sm, offset + onewire_offset_fetch_bit, &c);
87
88 // enable the state machine
89 pio_sm_set_enabled (pio, sm, true);
90 }
91

```

```
92 static inline uint onewire_reset_instr (uint offset) {
93     // encode a "jmp reset_bus side 0" instruction for the state machine
94     return pio_encode_jmp (offset + onewire_offset_reset_bus) | pio_encode_sideset (1, 0);
95 }
96 %}
```

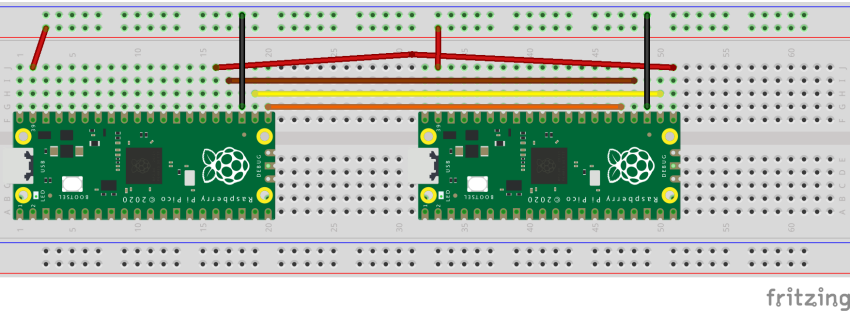
Communicating as master and slave via SPI

This example code shows how to interface two RP2040 microcontrollers to each other using SPI.

Wiring information

Function	Master (RP2040)	Slave (RP2040)	Master (Pico)	Slave (Pico)
MOSI	DO0	DI0	25	21
SCLK	SCK0	SCK0	24	24
GND	GND	GND	23	23
CS	CS0	CS0	22	22
MISO	DI0	DO0	21	25

Figure 28. Wiring Diagram for SPI Master and Slave.



At least one of the boards should be powered, and will share power to the other.

If the master is not connected properly to a slave, the master will report reading all zeroes.

If the slave is not connected properly to a master, it will initialize but never transmit nor receive, because it's waiting for clock signal from the master.

Outputs

Both master and slave boards will give output to stdio.

With master and slave properly connected, the master should output something like this:

```
SPI master example
SPI master says: The following buffer will be written to MOSI endlessly:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f
40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f
```

```

60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f
70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f
80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f
90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f
a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af
b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf
c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf
d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df
e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef
f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff
SPI master says: read page 0 from the MISO line:
ff fe fd fc fb fa f9 f8 f7 f6 f5 f4 f3 f2 f1 f0
ef ee ed ec eb ea e9 e8 e7 e6 e5 e4 e3 e2 e1 e0
df de dd dc db da d9 d8 d7 d6 d5 d4 d3 d2 d1 d0
cf ce cd cc cb ca c9 c8 c7 c6 c5 c4 c3 c2 c1 c0
bf be bd bc bb ba b9 b8 b7 b6 b5 b4 b3 b2 b1 b0
af ae ad ac ab aa a9 a8 a7 a6 a5 a4 a3 a2 a1 a0
9f 9e 9d 9c 9b 9a 99 98 97 96 95 94 93 92 91 90
8f 8e 8d 8c 8b 8a 89 88 87 86 85 84 83 82 81 80
7f 7e 7d 7c 7b 7a 79 78 77 76 75 74 73 72 71 70
6f 6e 6d 6c 6b 6a 69 68 67 66 65 64 63 62 61 60
5f 5e 5d 5c 5b 5a 59 58 57 56 55 54 53 52 51 50
4f 4e 4d 4c 4b 4a 49 48 47 46 45 44 43 42 41 40
3f 3e 3d 3c 3b 3a 39 38 37 36 35 34 33 32 31 30
2f 2e 2d 2c 2b 2a 29 28 27 26 25 24 23 22 21 20
1f 1e 1d 1c 1b 1a 19 18 17 16 15 14 13 12 11 10
0f 0e 0d 0c 0b 0a 09 08 07 06 05 04 03 02 01 00

```

The slave should output something like this:

```

SPI slave example
SPI slave says: When reading from MOSI, the following buffer will be written to MISO:
ff fe fd fc fb fa f9 f8 f7 f6 f5 f4 f3 f2 f1 f0
ef ee ed ec eb ea e9 e8 e7 e6 e5 e4 e3 e2 e1 e0
df de dd dc db da d9 d8 d7 d6 d5 d4 d3 d2 d1 d0
cf ce cd cc cb ca c9 c8 c7 c6 c5 c4 c3 c2 c1 c0
bf be bd bc bb ba b9 b8 b7 b6 b5 b4 b3 b2 b1 b0
af ae ad ac ab aa a9 a8 a7 a6 a5 a4 a3 a2 a1 a0
9f 9e 9d 9c 9b 9a 99 98 97 96 95 94 93 92 91 90
8f 8e 8d 8c 8b 8a 89 88 87 86 85 84 83 82 81 80
7f 7e 7d 7c 7b 7a 79 78 77 76 75 74 73 72 71 70
6f 6e 6d 6c 6b 6a 69 68 67 66 65 64 63 62 61 60
5f 5e 5d 5c 5b 5a 59 58 57 56 55 54 53 52 51 50
4f 4e 4d 4c 4b 4a 49 48 47 46 45 44 43 42 41 40
3f 3e 3d 3c 3b 3a 39 38 37 36 35 34 33 32 31 30
2f 2e 2d 2c 2b 2a 29 28 27 26 25 24 23 22 21 20
1f 1e 1d 1c 1b 1a 19 18 17 16 15 14 13 12 11 10
0f 0e 0d 0c 0b 0a 09 08 07 06 05 04 03 02 01 00
SPI slave says: read page 0 from the MOSI line:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f
40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f
60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f
70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f
80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f
90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f
a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af
b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf
c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf

```

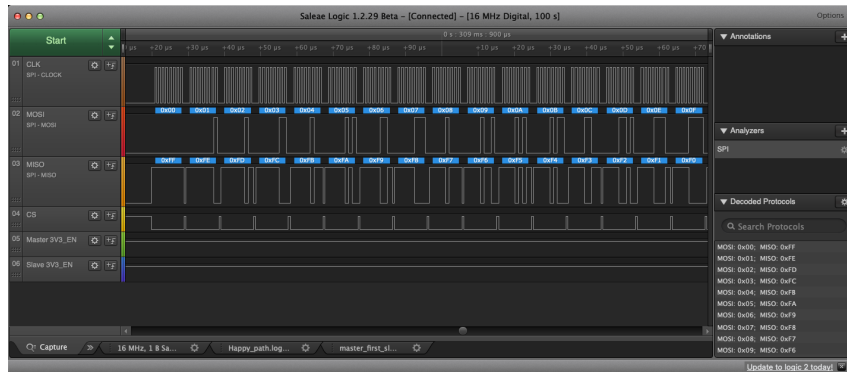
```

d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df
e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef
f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff

```

If you look at the communication with a logic analyzer, you should see this:

Figure 29. Data capture as seen in Saleae Logic.



List of Files

CMakeLists.txt

CMake file to incorporate the example in to the examples build tree.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/spi_master_slave/CMakeLists.txt

```

1 add_subdirectory_exclude_platforms(spi_master)
2 add_subdirectory_exclude_platforms(spi_slave)

```

spi_master/spi_master.c

The example code for SPI master.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/spi_master_slave/spi_master/spi_master.c

```

1 // Copyright (c) 2021 Michael Stoops. All rights reserved.
2 // Portions copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3 //
4 // Redistribution and use in source and binary forms, with or without modification, are
5 // permitted provided that the
6 // following conditions are met:
7 //
8 // 1. Redistributions of source code must retain the above copyright notice, this list of
9 // conditions and the following
10 // disclaimer.
11 // 2. Redistributions in binary form must reproduce the above copyright notice, this list of
12 // conditions and the
13 // following disclaimer in the documentation and/or other materials provided with the
14 // distribution.
15 // 3. Neither the name of the copyright holder nor the names of its contributors may be used to
16 // endorse or promote
17 // products derived from this software without specific prior written permission.
18 //
19 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS
20 // OR IMPLIED WARRANTIES,
21 // INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
22 // PARTICULAR PURPOSE ARE
23 // DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,

```

```

    INDIRECT, INCIDENTAL,
17 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
    SUBSTITUTE GOODS OR
18 // SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
    THEORY OF LIABILITY,
19 // WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING
    IN ANY WAY OUT OF THE
20 // USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
21 //
22 // SPDX-License-Identifier: BSD-3-Clause
23 //
24 // Example of an SPI bus master using the PL022 SPI interface
25
26 #include <stdio.h>
27 #include "pico/stdlib.h"
28 #include "pico/binary_info.h"
29 #include "hardware/spi.h"
30
31 #define BUF_LEN      0x100
32
33 void printbuf(uint8_t buf[], size_t len) {
34     size_t i;
35     for (i = 0; i < len; ++i) {
36         if (i % 16 == 15)
37             printf("%02x\n", buf[i]);
38         else
39             printf("%02x ", buf[i]);
40     }
41
42     // append trailing newline if there isn't one
43     if (i % 16) {
44         putchar('\n');
45     }
46 }
47
48 int main() {
49     // Enable UART so we can print
50     stdio_init_all();
51     #if !defined(spi_default) || !defined(PICO_DEFAULT_SPI_SCK_PIN) ||
    !defined(PICO_DEFAULT_SPI_TX_PIN) || !defined(PICO_DEFAULT_SPI_RX_PIN) ||
    !defined(PICO_DEFAULT_SPI_CSN_PIN)
52     #warning spi/spi_master example requires a board with SPI pins
53     puts("Default SPI pins were not defined");
54     #else
55
56     printf("SPI master example\n");
57
58     // Enable SPI 0 at 1 MHz and connect to GPIOs
59     spi_init(spi_default, 1000 * 1000);
60     gpio_set_function(PICO_DEFAULT_SPI_RX_PIN, GPIO_FUNC_SPI);
61     gpio_set_function(PICO_DEFAULT_SPI_SCK_PIN, GPIO_FUNC_SPI);
62     gpio_set_function(PICO_DEFAULT_SPI_TX_PIN, GPIO_FUNC_SPI);
63     gpio_set_function(PICO_DEFAULT_SPI_CSN_PIN, GPIO_FUNC_SPI);
64     // Make the SPI pins available to picotool
65     bi_decl(bi_4pins_with_func(PICO_DEFAULT_SPI_RX_PIN, PICO_DEFAULT_SPI_TX_PIN,
    PICO_DEFAULT_SPI_SCK_PIN, PICO_DEFAULT_SPI_CSN_PIN, GPIO_FUNC_SPI));
66
67     uint8_t out_buf[BUF_LEN], in_buf[BUF_LEN];
68
69     // Initialize output buffer
70     for (size_t i = 0; i < BUF_LEN; ++i) {
71         out_buf[i] = i;
72     }
73

```

```

74     printf("SPI master says: The following buffer will be written to MOSI endlessly:\n");
75     printf(out_buf, BUF_LEN);
76
77     for (size_t i = 0; ; ++i) {
78         // Write the output buffer to MOSI, and at the same time read from MISO.
79         spi_write_read_blocking(spi_default, out_buf, in_buf, BUF_LEN);
80
81         // Write to stdio whatever came in on the MISO line.
82         printf("SPI master says: read page %d from the MISO line:\n", i);
83         printf(in_buf, BUF_LEN);
84
85         // Sleep for ten seconds so you get a chance to read the output.
86         sleep_ms(10 * 1000);
87     }
88 #endif
89 }

```

spi_slave/spi_slave.c

The example code for SPI slave.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/spi/spi_master_slave/spi_slave/spi_slave.c

```

1  // Copyright (c) 2021 Michael Stoops. All rights reserved.
2  // Portions copyright (c) 2021 Raspberry Pi (Trading) Ltd.
3  //
4  // Redistribution and use in source and binary forms, with or without modification, are
5  // permitted provided that the
6  // following conditions are met:
7  //
8  // 1. Redistributions of source code must retain the above copyright notice, this list of
9  // conditions and the following
10 // disclaimer.
11 // 2. Redistributions in binary form must reproduce the above copyright notice, this list of
12 // conditions and the
13 // following disclaimer in the documentation and/or other materials provided with the
14 // distribution.
15 // 3. Neither the name of the copyright holder nor the names of its contributors may be used to
16 // endorse or promote
17 // products derived from this software without specific prior written permission.
18 //
19 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS
20 // OR IMPLIED WARRANTIES,
21 // INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
22 // PARTICULAR PURPOSE ARE
23 // DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
24 // INDIRECT, INCIDENTAL,
25 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
26 // SUBSTITUTE GOODS OR
27 // SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
28 // THEORY OF LIABILITY,
29 // WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING
30 // IN ANY WAY OUT OF THE
31 // USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
32 //
33 // SPDX-License-Identifier: BSD-3-Clause
34 //
35 // Example of an SPI bus slave using the PL022 SPI interface
36
37 #include <stdio.h>
38 #include <string.h>
39 #include "pico/stdlib.h"
40 #include "pico/binary_info.h"

```

```

30 #include "hardware/spi.h"
31
32 #define BUF_LEN      0x100
33
34 void printbuf(uint8_t buf[], size_t len) {
35     size_t i;
36     for (i = 0; i < len; ++i) {
37         if (i % 16 == 15)
38             printf("%02x\n", buf[i]);
39         else
40             printf("%02x ", buf[i]);
41     }
42
43     // append trailing newline if there isn't one
44     if (i % 16) {
45         putchar('\n');
46     }
47 }
48
49
50 int main() {
51     // Enable UART so we can print
52     stdio_init_all();
53     #if !defined(spi_default) || !defined(PICO_DEFAULT_SPI_SCK_PIN) ||
54         !defined(PICO_DEFAULT_SPI_TX_PIN) || !defined(PICO_DEFAULT_SPI_RX_PIN) ||
55         !defined(PICO_DEFAULT_SPI_CSN_PIN)
56     #warning spi/spi_slave example requires a board with SPI pins
57     puts("Default SPI pins were not defined");
58 #else
59
60     printf("SPI slave example\n");
61
62     // Enable SPI 0 at 1 MHz and connect to GPIOs
63     spi_init(spi_default, 1000 * 1000);
64     spi_set_slave(spi_default, true);
65     gpio_set_function(PICO_DEFAULT_SPI_RX_PIN, GPIO_FUNC_SPI);
66     gpio_set_function(PICO_DEFAULT_SPI_SCK_PIN, GPIO_FUNC_SPI);
67     gpio_set_function(PICO_DEFAULT_SPI_TX_PIN, GPIO_FUNC_SPI);
68     gpio_set_function(PICO_DEFAULT_SPI_CSN_PIN, GPIO_FUNC_SPI);
69     // Make the SPI pins available to picotool
70     bi_decl(bi_4pins_with_func(PICO_DEFAULT_SPI_RX_PIN, PICO_DEFAULT_SPI_TX_PIN,
71     PICO_DEFAULT_SPI_SCK_PIN, PICO_DEFAULT_SPI_CSN_PIN, GPIO_FUNC_SPI));
72
73     uint8_t out_buf[BUF_LEN], in_buf[BUF_LEN];
74
75     // Initialize output buffer
76     for (size_t i = 0; i < BUF_LEN; ++i) {
77         // bit-inverted from i. The values should be: {0xff, 0xfe, 0xfd...}
78         out_buf[i] = ~i;
79     }
80
81     printf("SPI slave says: When reading from MOSI, the following buffer will be written to MISO:\n");
82     printbuf(out_buf, BUF_LEN);
83
84     for (size_t i = 0; ; ++i) {
85         // Write the output buffer to MISO, and at the same time read from MOSI.
86         spi_write_read_blocking(spi_default, out_buf, in_buf, BUF_LEN);
87
88         // Write to stdio whatever came in on the MOSI line.
89         printf("SPI slave says: read page %d from the MOSI line:\n", i);
90         printbuf(in_buf, BUF_LEN);
91     }
92 #endif

```



```
90 }
```

Bill of Materials

Table 54. A list of materials required for the example

Item	Quantity	Details
Breadboard	1	generic part
Raspberry Pi Pico	2	https://www.raspberrypi.com/products/raspberry-pi-pico/
M/M Jumper wires	8	generic part